

차원 축소 & 비지도 학습

한국화학연구원
화학데이터기반연구센터
장 승 훈

교육 내용 및 순서

- 이론

- ✓ 비지도 학습 - K-means(준지도학습), DBSCAN
- ✓ 차원축소법 - PCA, LLE, 오토인코더 (NN 설명 포함)

- 실습

- ✓ K-means를 이용한 색상 분할, 전처리, 준지도 학습
- ✓ PCA 실습
- ✓ ChemAI - DBSCAN 실습

비지도 학습 K-means, DBSCAN

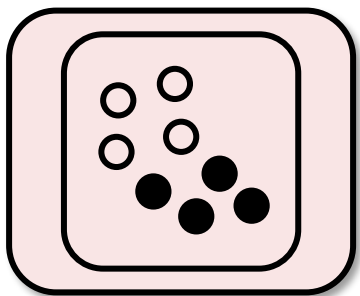
머신러닝 분류

머신러닝 시스템을

“학습하는 동안의 정보량(또는 감독의 유무)”에 따라 분류할 수 있음

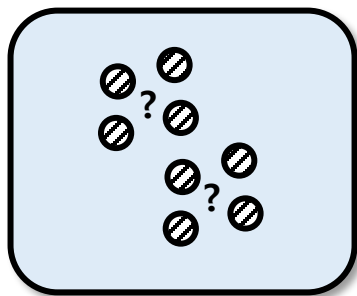
지도학습

분류
회귀
인공신경망



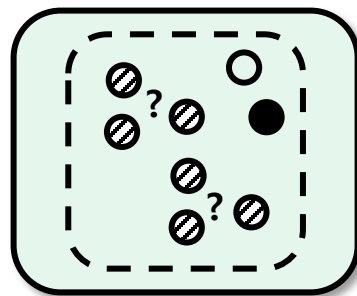
비지도학습

군집
시각화
차원축소



준지도학습

DBM
RBM



강화학습

Reinforcement
learning

지도/**비지도**/강화 학습



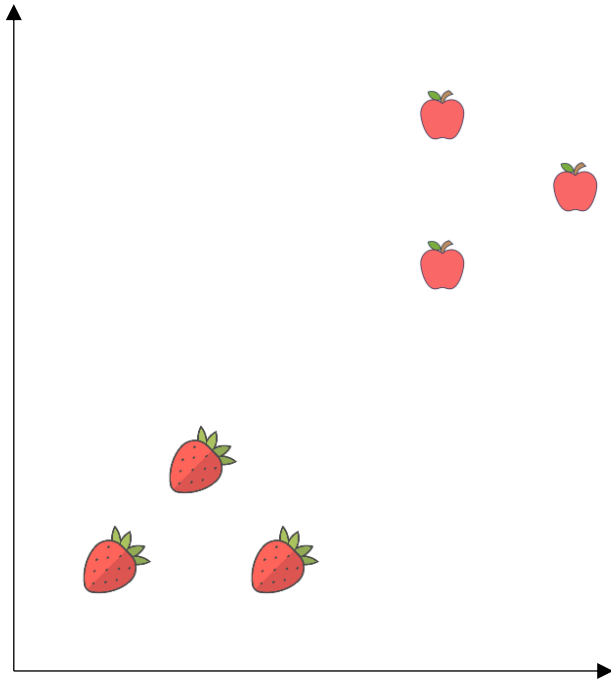
강화 학습 (체리, 극히 일부)

지도 학습 (케익 겉면 크림, 레이
블된 데이터양이 적음.)

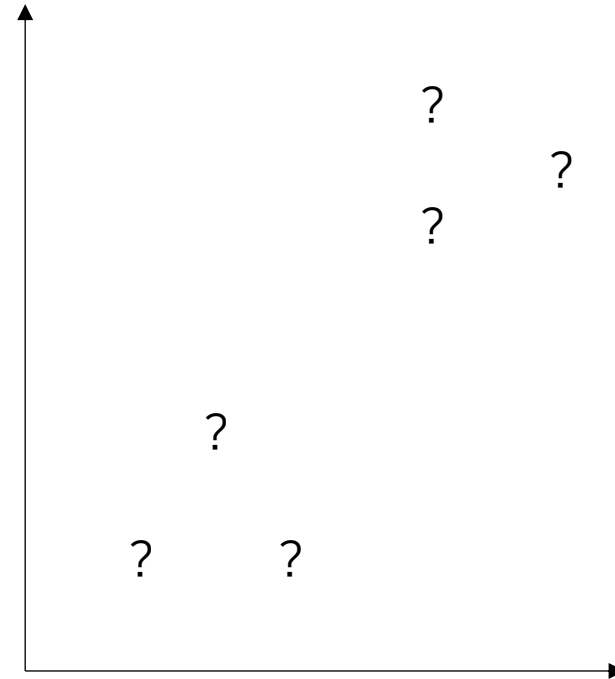
**비지도 학습 (케익 빵, 잠재력[데이
터량]이 큼, 안쪽[핵심]부분)**

CNN구조를 창시한 얀 르쿤 교수(메타(Meta)의 인공지능(AI) 연구소 수석 과학자, 뉴욕 대학 교수, 현직)의 머신러닝의 각 학습 방법에 대한 설명 인용.

지도학습 vs 비지도학습



지도학습
Supervised Learning



비지도학습
Unsupervised Learning

지도학습 vs 비지도학습

맛	모양	과일이름
3	1	딸기
5	4	사과
1	1	딸기
2	2	딸기
4	6	사과
5	5	사과

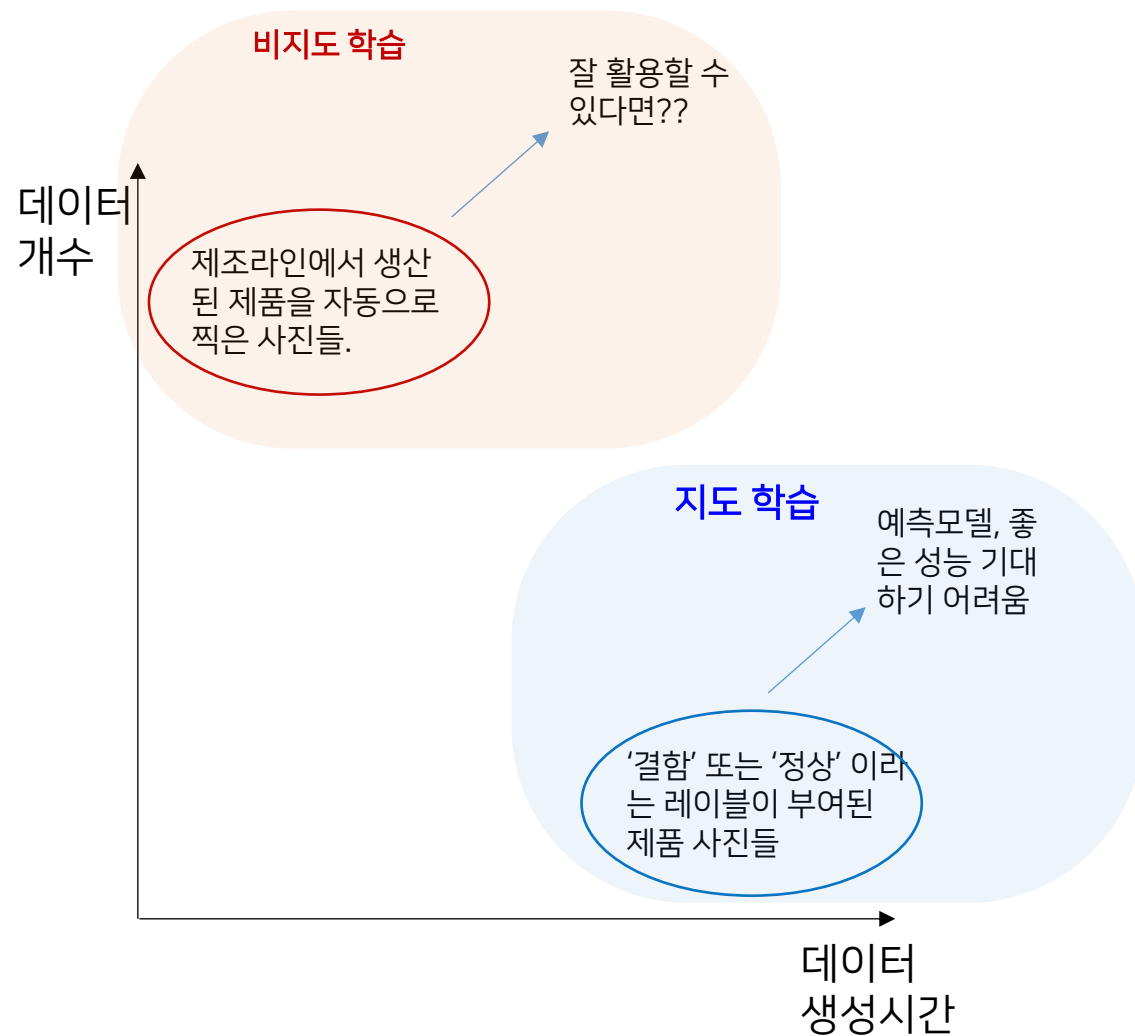
지도학습
Supervised Learning

맛	모양
3	1
5	4
1	1
2	2
4	6
5	5

비지도학습
Unsupervised Learning

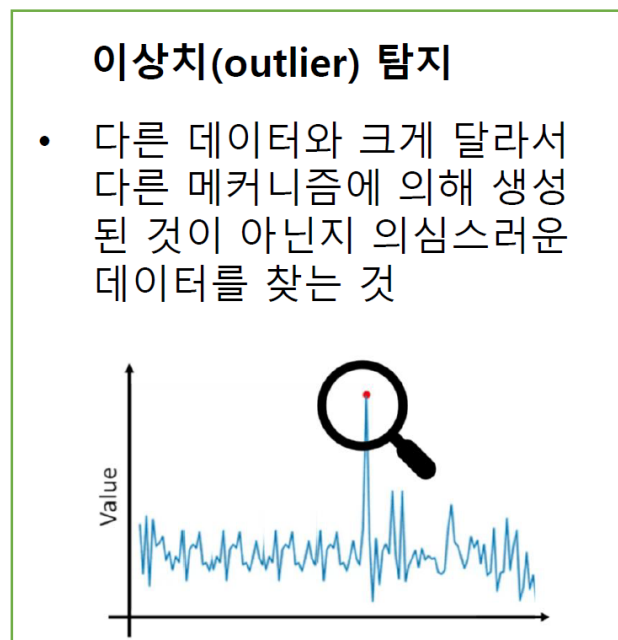
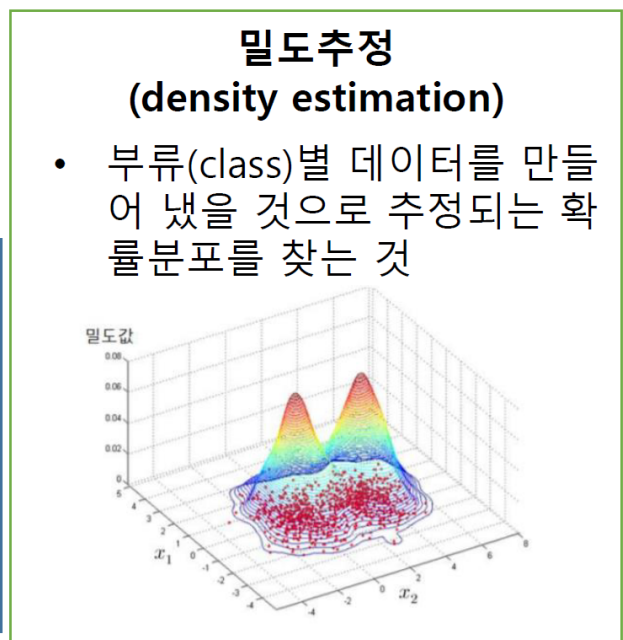
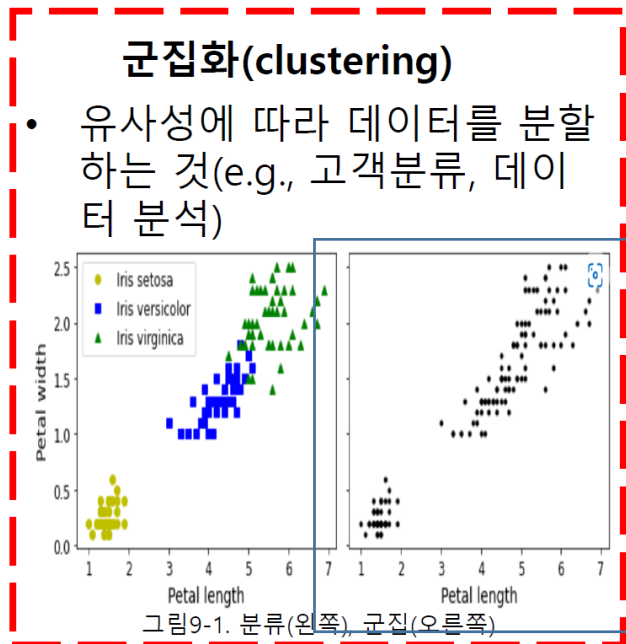
- Dataset
- # of data
- Target data
- Feature data
- Classification (과일이름)
- Regression (가격예측)

지도 학습 vs 비지도 학습



비지도 학습

비지도학습 : 결과정보 (label) 가 없는 데이터들에 대해서 특정 패턴을 찾는 것.



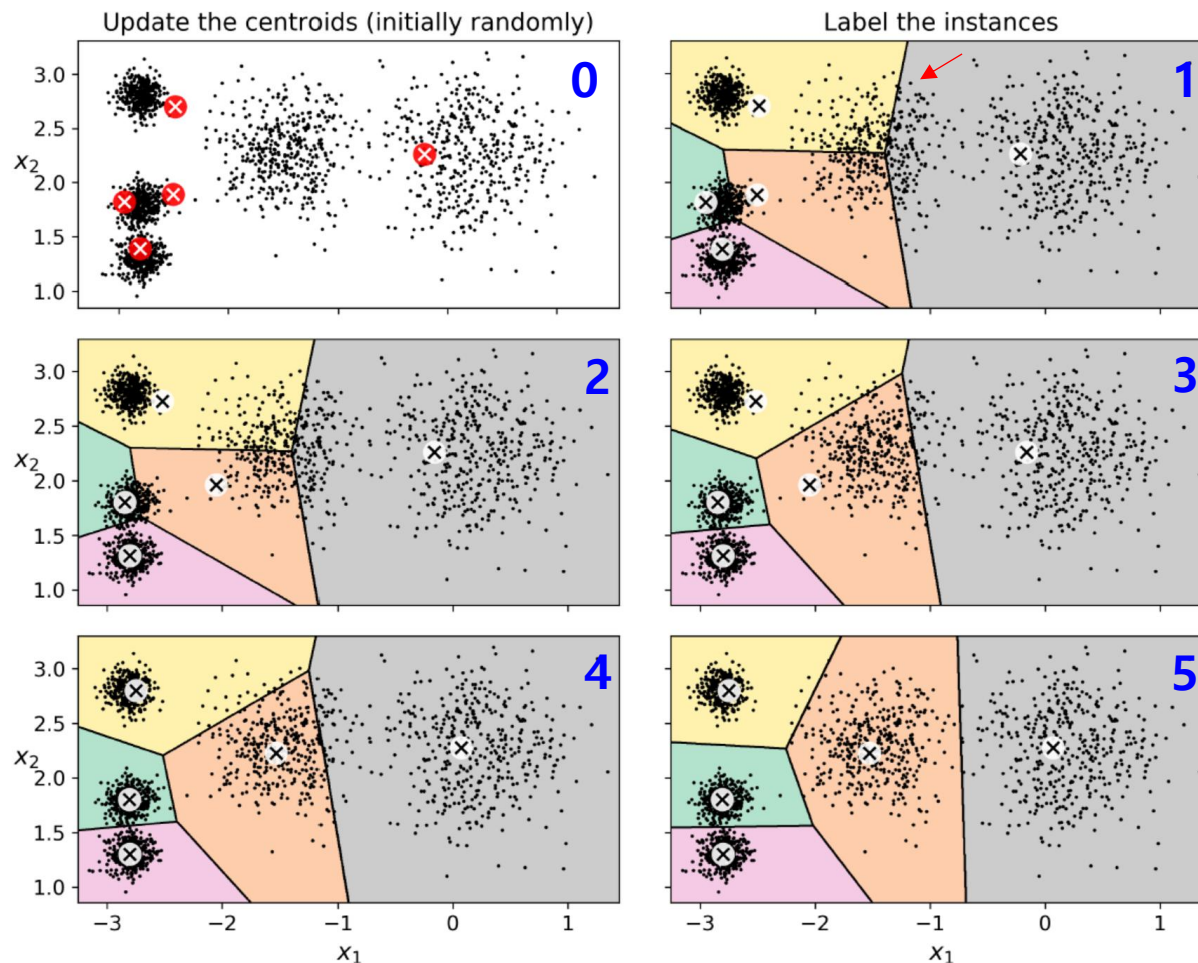
Ref.
<https://gaebom.tistory.com/6>
<https://velog.io/@euisuk-chung>

Clustering에 대해서 자세히 살펴보자!

K-means Clustering

K-means 알고리즘 설명

무작위(initially randomly)로
샘플을 뽑아 해당 위치를 **센트로**
이드로 지정.



샘플과 가장 가까운 센트로이드
사이의 평균 제곱 거리(이너셔,
inertia)를 계산.

샘플들에 레이블을 할당하고 센
트로이드를 업데이트하고

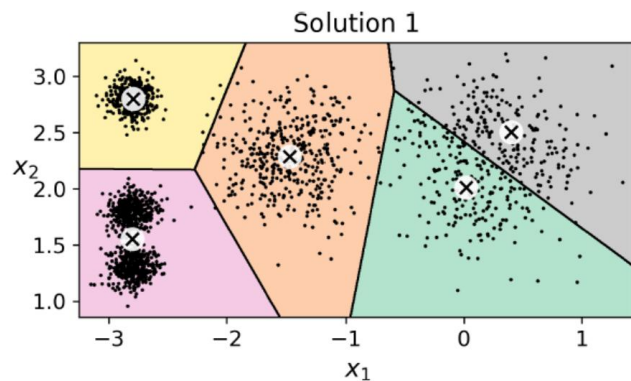
샘플에 레이블을 할당하고
센트로이드를 업데이트하고..

센트로이드 변화가 없을 때까지
반복.

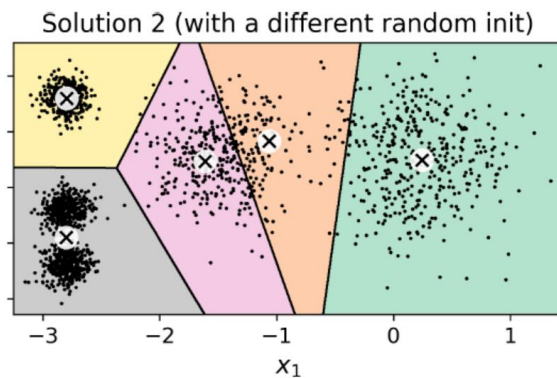
The K-Means algorithm

K-means : 이너셔

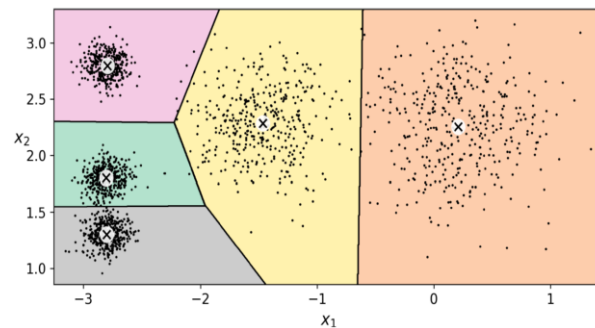
◆ 이너셔(inertia) : 각 샘플과 가장 가까운 센트로이드 사이의 평균 제곱거리.



Inertia = 223.3



Inertia = 223.3



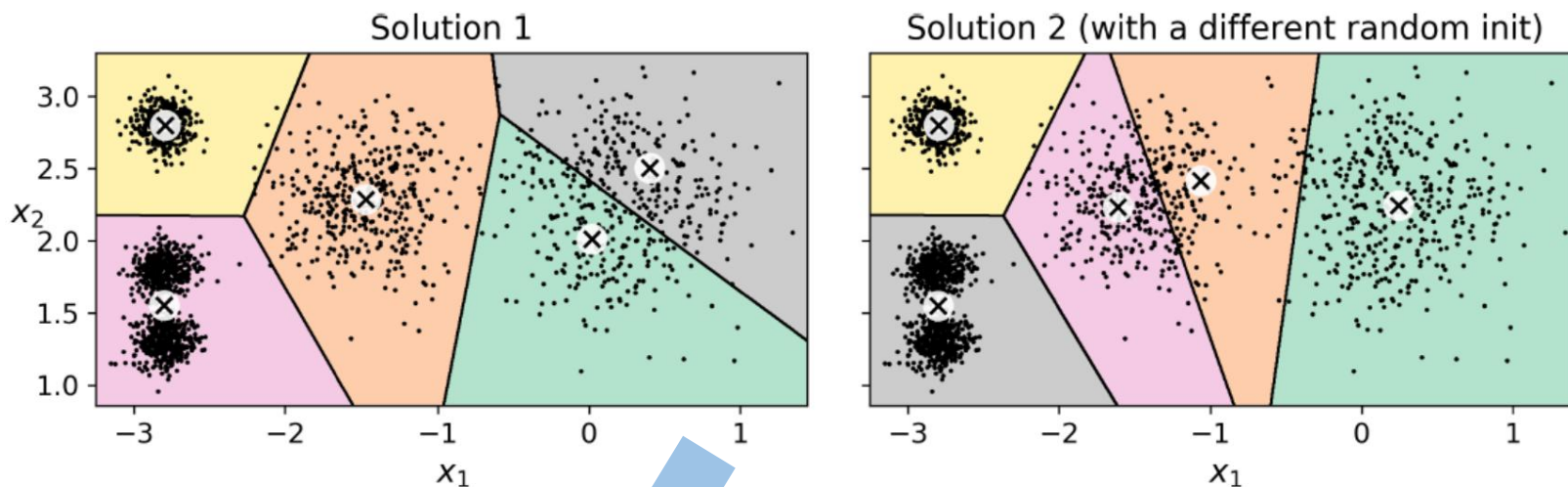
Inertia = 211.6

이너셔(inertia) 값이 같아도 전혀 다르게 레이블이 할당될 수 있음!!

K-means

랜덤 초기화의 한계점 존재.

- 운이 없을 경우, k-means 알고리즘은 최적이지 아닌 솔루션을 제시할 수 있음.



문제 해결 방법

센트로이드 위치를 어느정도 알 수 있다면 센트로이드 리스트를 담은 넘파이 배열을 지정하여 init 에 할당

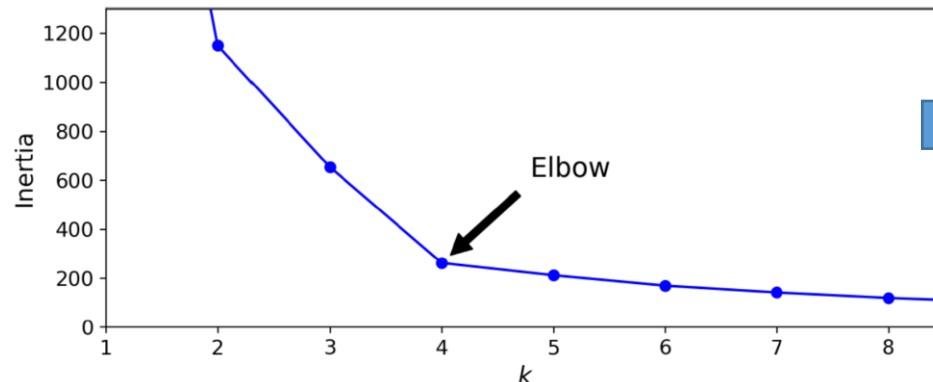
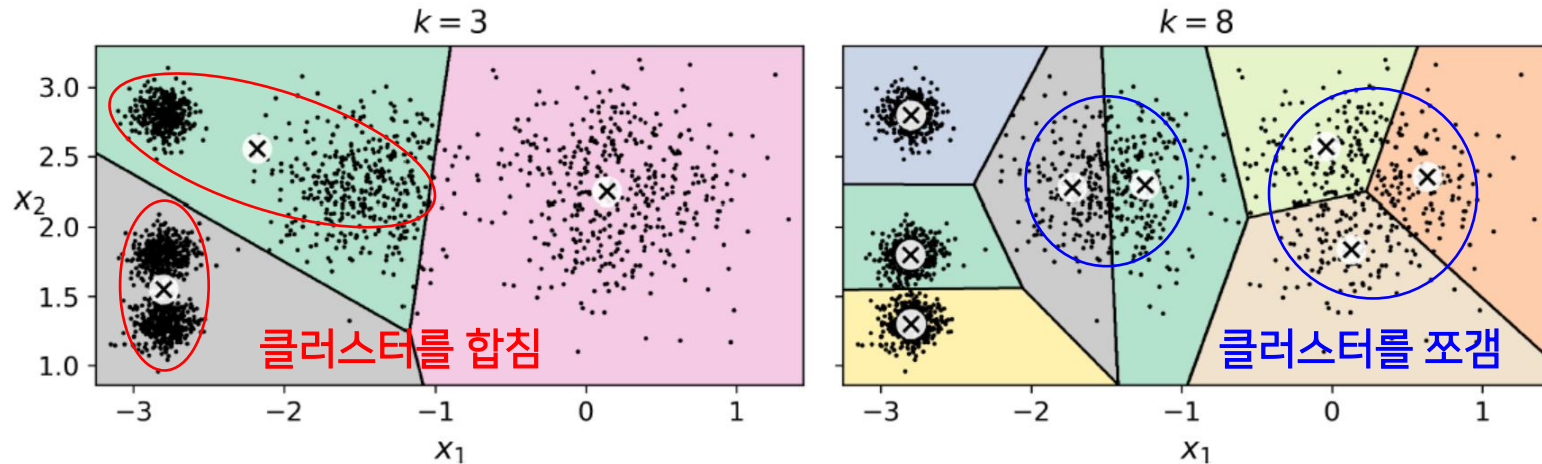
n_init : 랜덤 초기화 횟수 (기본값:10)
: 이너셔가 가장 낮은 모델을 반환

```
good_init = np.array([[ -3, 3], [ -3, 2], [ -3, 1], [ -1, 2], [ 0, 2]])  
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1)
```

K-means : k값 결정

최적의 k 값 찾기

- 최적의 k 개수는 모델 성능을 결정짓는 중요한 요소
- 아래 데이터의 경우 최적의 k값이 5라는 사실을 알 수 있음.
- 그러나 대부분의 경우, k값을 얼마로 설정해야 할지 알 수 없음.



이너셔를 k 의 함수로 그린 그림.

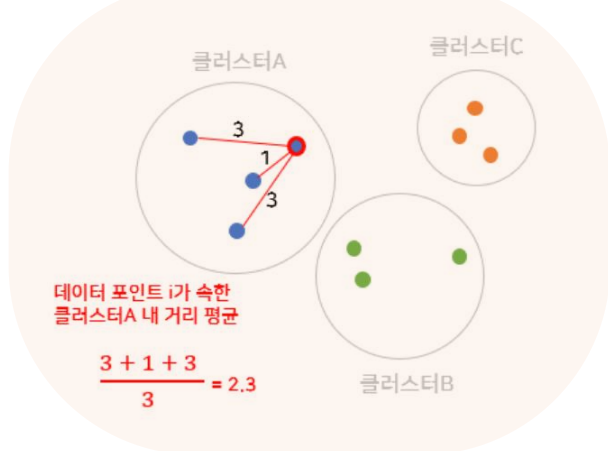
- 이너셔는 k 가 4까지 빠르게 감소.
- 4 이상부터는 k 가 증가해도 이너셔 감소가 미미.
- 만약 정답을 모르는 상황이라면 $k=4$ 가 가장 좋은 선택.
- 이런 방식의 k 선택법은 조금 영성(靈性)한 부분이 있음.

K-means : K값 결정

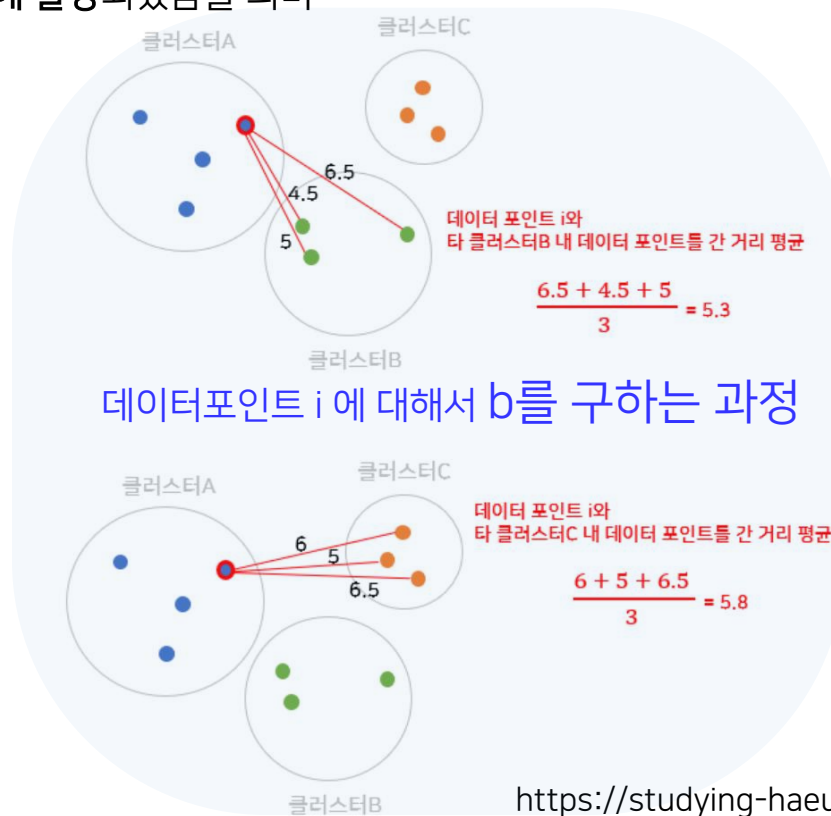
실루엣 점수 (silhouette score)

- 모든 샘플에 대한 실루엣 계수의 평균값.
- 실루엣 계수는 각 샘플에 대해서 $(b-a)/\max(a,b)$ 로 계산.
- a 는 "클러스터 내부의 평균 거리", b 는 "가장 가까운 클러스터까지의 평균 거리".
- 실루엣 계수는 -1부터 1까지 바뀔 수 있음.
- "1"에 가까우면 자신의 클러스터 안에 잘 속해 있고, "0"에 가까우면 클러스터 경계에 위치
- "-1"에 가까우면 잘못된 클러스터에 할당되었음을 의미

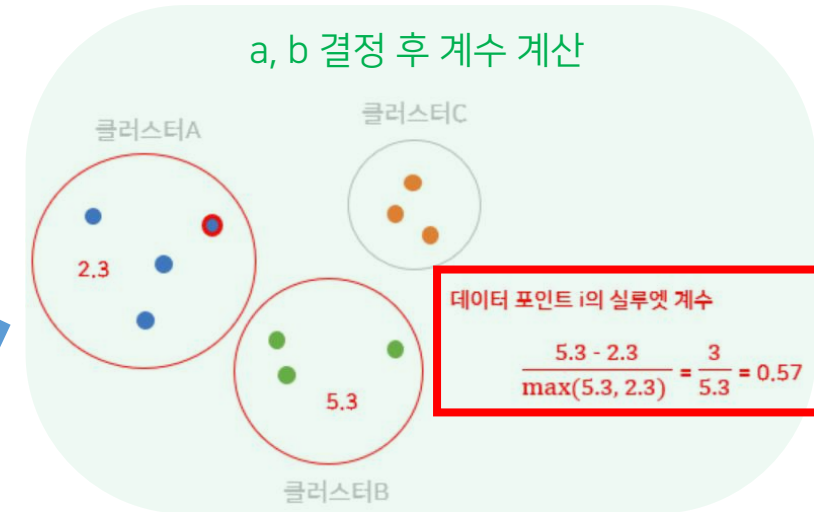
데이터포인트 i에 대해서
 a 를 구하는 과정



데이터포인트 i에 대해서 b 를 구하는 과정



a, b 결정 후 계수 계산



모든 포인트에 대한
평균값 계산

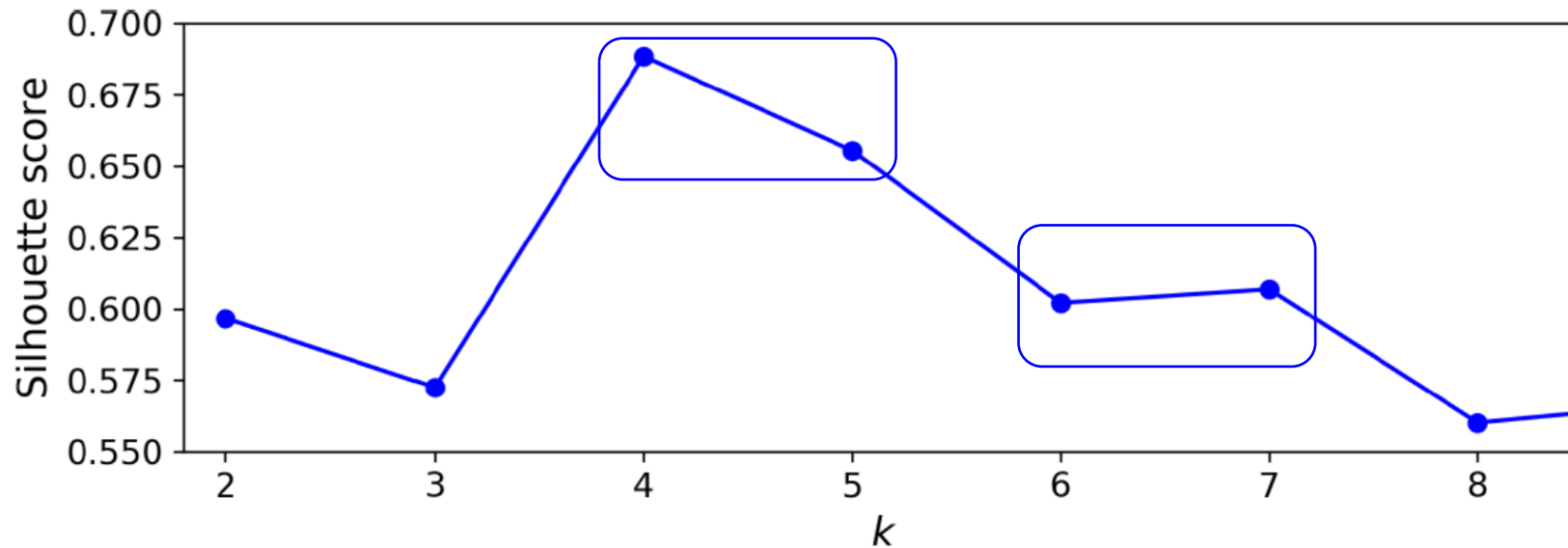


K-means : K값 결정

실루엣 점수 (silhouette score)

Kmeans 를 통해서 훈련
된 샘플들의 레이블 정보

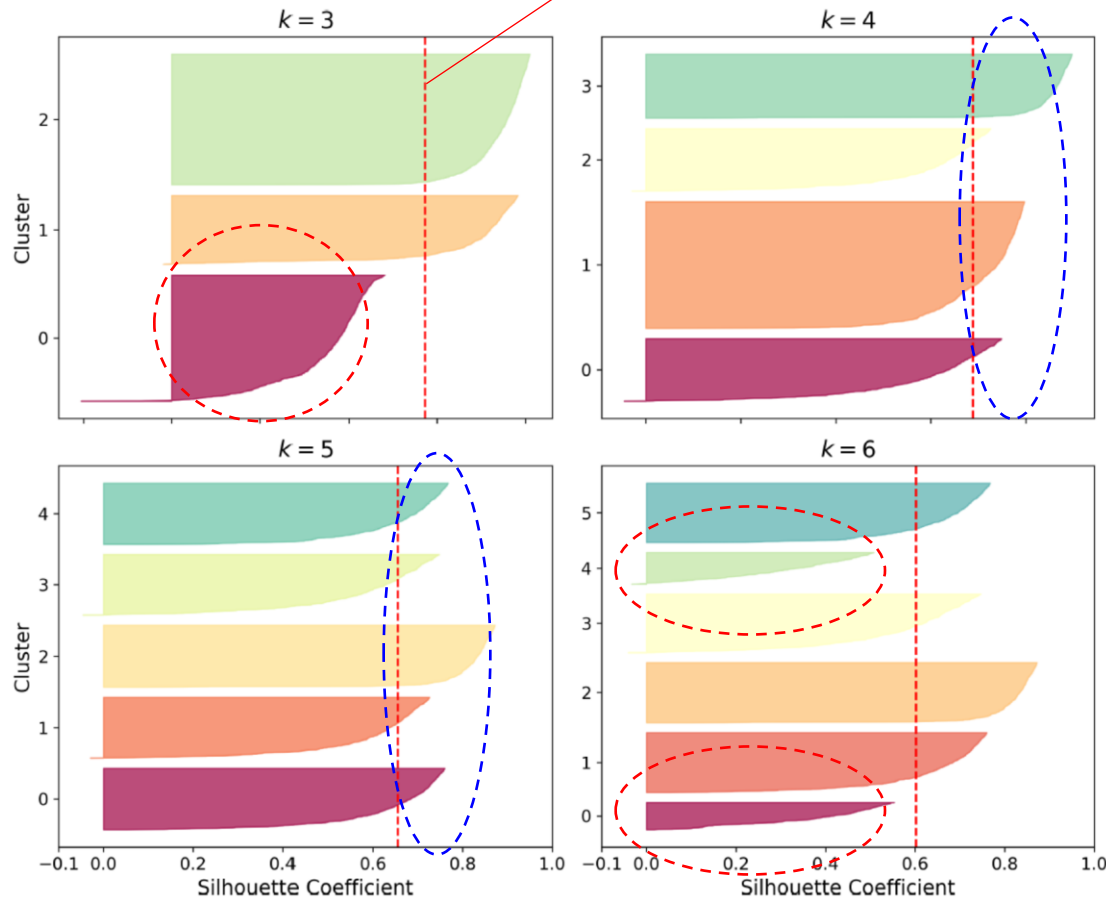
```
>>> from sklearn.metrics import silhouette_score  
>>> silhouette_score(X, kmeans.labels_)  
0.655517642572828
```



k=4, 5 인 경우 k=6, 7인 경우에 비해서 훨씬 좋은 실루엣 점수를 나타냄.
이러한 차이는 이너셔에 대한 분석에서는 드러나지 않았음.

K-means : K값 결정

수직 파선은 각 클러스터 개수에
해당하는 실루엣 점수를 표시한 것.



실루엣 다이어그램.

모든 샘플의 실루엣 계수를 할당된 클러스터와 계수값을
정렬하여 그린 그림.

칼 모양의 그래프.

높이 : 클러스터가 포함하고 있는 샘플의 개수를 의미
너비 : 클러스터에 포함된 샘플들의 정렬된 실루엣 계수
를 나타냄. (넓을수록, 1에 가까울 수록 좋음)

실루엣 다이어그램 상에서도 k=4, 5의 경우가 좋아보임.

굳이, k=4, k=5를 비교해보면, k=5일 때 비슷한 크기의
클러스터를 얻을 수 있으므로, 실루엣 점수가 조금 낮더
라도 k=5를 선택하는 것이 좋음.

K-means : 한계

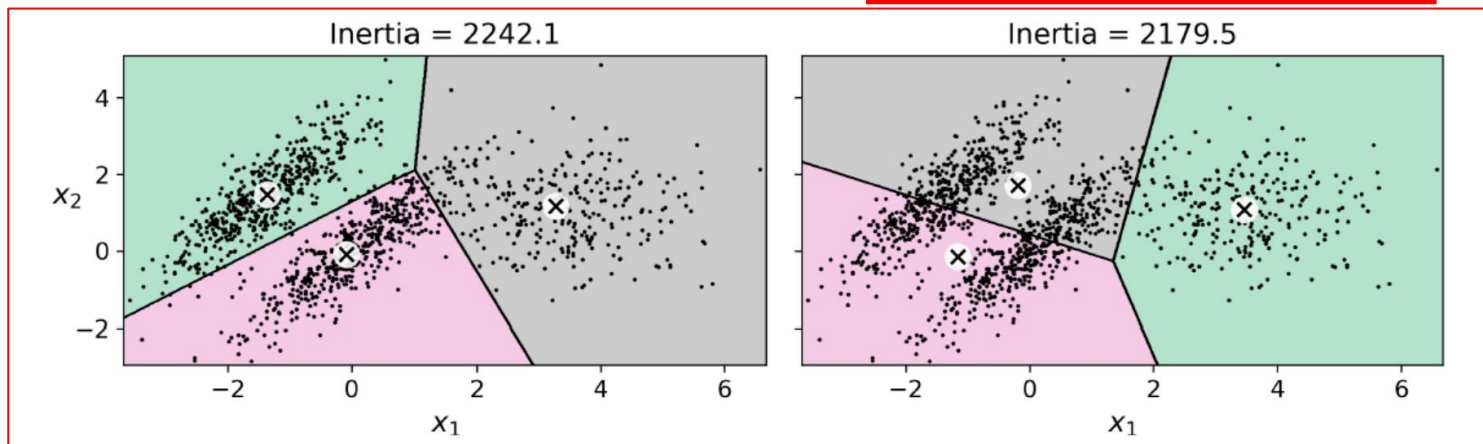
K-means의 한계

- 장점:

- 1) 속도가 빠르고 확장성이 용이함
- 2) 데이터에 대한 사전 정보가 필요하지 않음

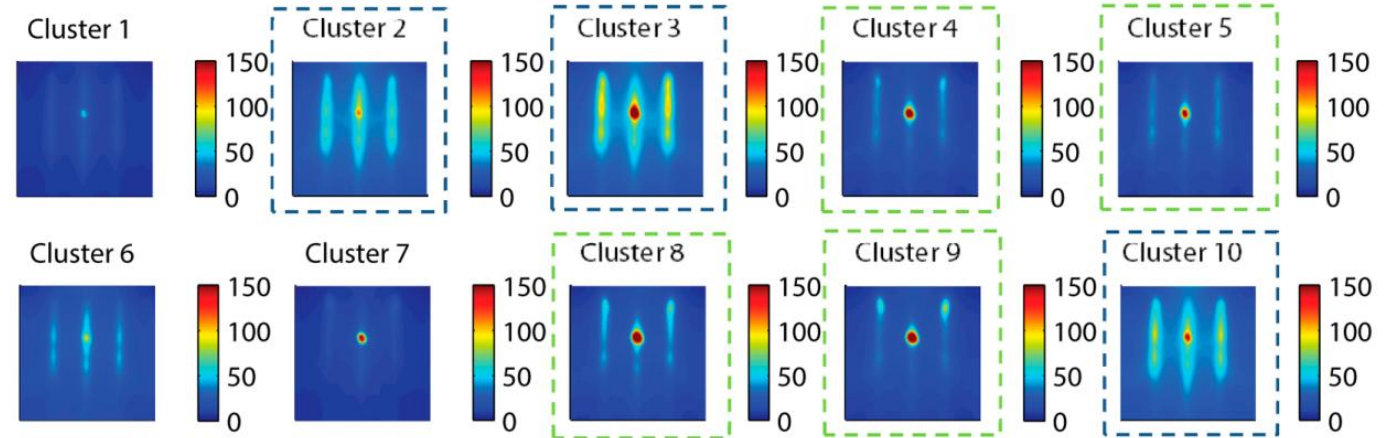
- 단점:

- 1) 최적이 아닌 솔루션을 피하려면 알고리즘을 여러 번 실행해야 함
- 2) 초기 클러스터링 수 결정하는데 어려움이 있음
- 3) 클러스터 수가 적합하지 않으면 결과해석의 어려움
- 4) 군집된 데이터 크기나 밀집도가 서로 다르거나 원형이 아닐 경우 잘 동작하지 않음



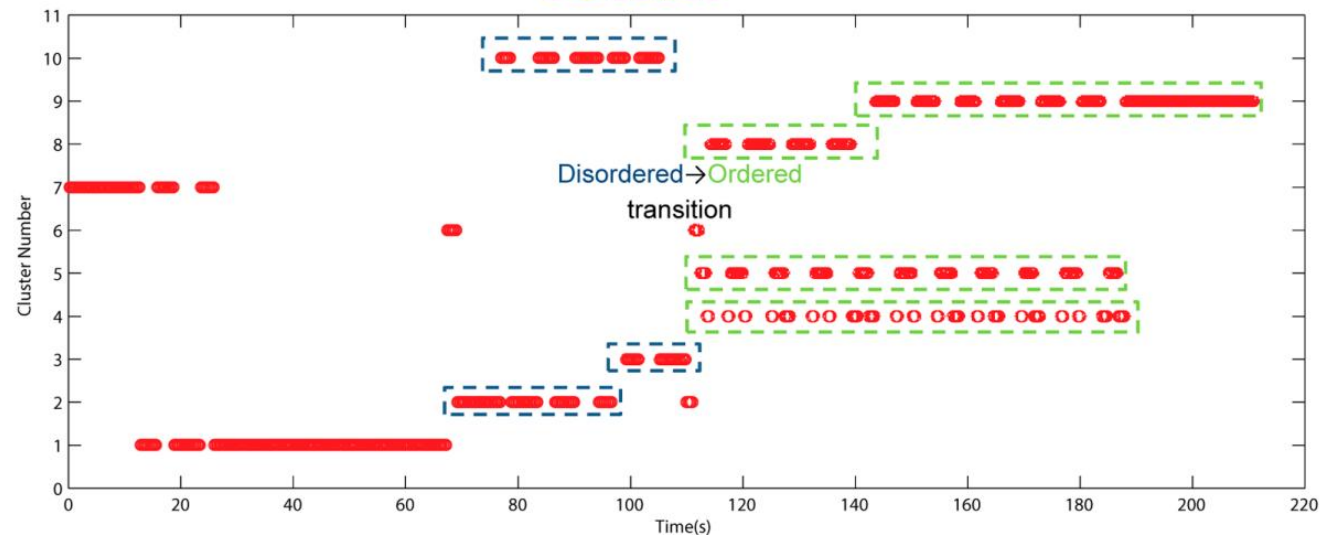
K-means를 이용한 소재 연구 예시 (1)

The image obtained by averaging the members of each cluster



Big-Data Reflection High Energy Electron Diffraction (RHEED) Analysis for Understanding Epitaxial Film Growth Processes
ACS Nano 2014, 8, 10, 10899–10908

Growth
mechanism



The timeline of the clusters

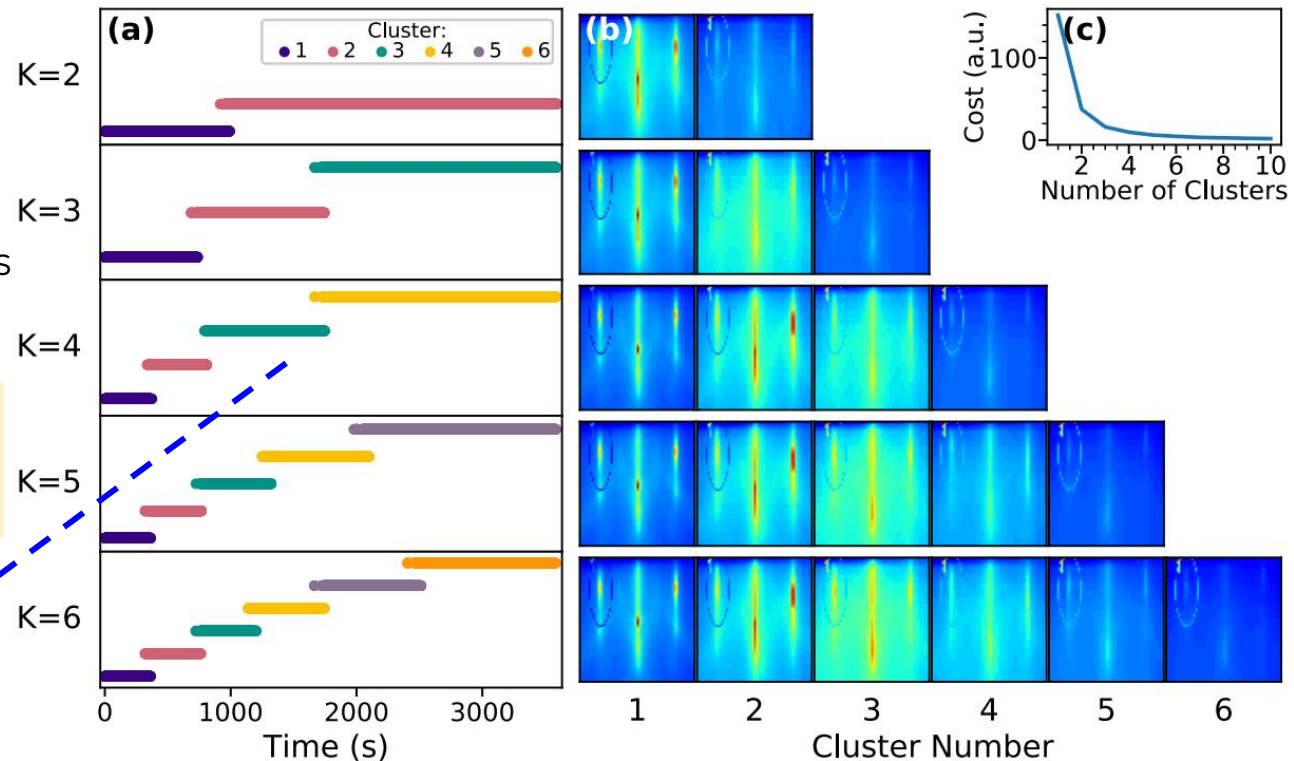
K-means를 이용한 소재 연구 예시 (2)

SrTiO₃(STO) homoepitaxial growth Analysis

→ STO₃ films were grown on (100) STO substrates

Machine learning analysis of perovskite oxides
grown by molecular beam epitaxy
PHYSICAL REVIEW MATERIALS 4, 083807 (2020)

해당분야 domain knowledge
와 비교해보고 타당한 설명이
가능한 K 값 선택.



(a) K-means clustering for K = 1-6 clusters, (b) the mean image of each cluster, and (c) the k-means minimization function plotted for each value of K for STO3

K-means를 이용한 소재 연구 예시 (3)

Growth mode analysis

Distinct thin film growth characteristics determined through comparative dimension reduction techniques
J. Appl. Phys. 130, 125301 (2021)

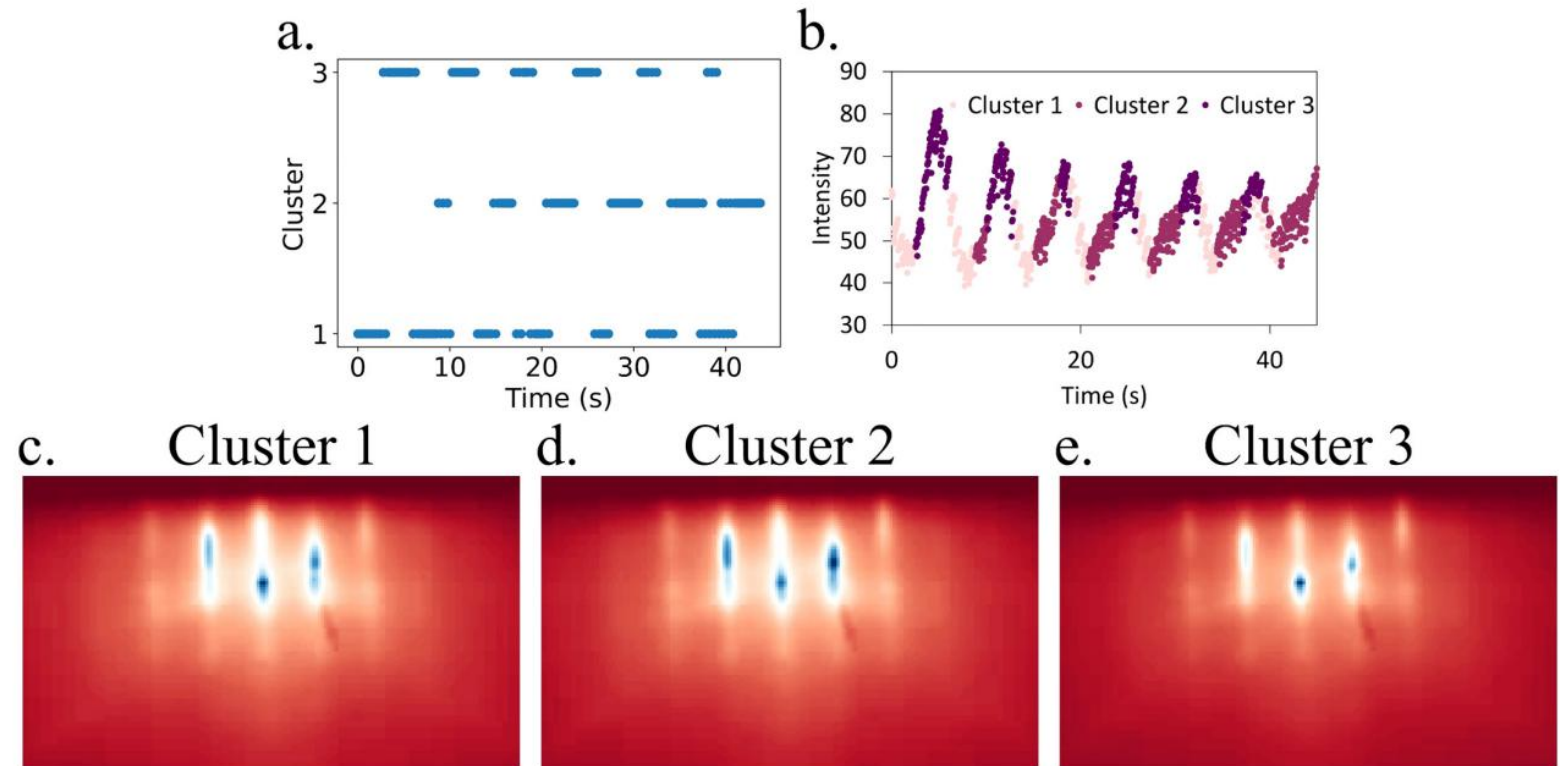


FIG. 4. Kmeans with three clusters. (a) The clusters with respect to time. (b) The clusters plotted over the intensity fluctuations of the (00) spot. The intensity was not scaled in this case. (c)–(e) The averages of images from clusters 1–3, respectively.

군집을 이용한 색상 분할

색상(color segmentation) 분할

- 예시 : 인공위성 사진을 분석하여 한 지역의 전체 산림 면적이 얼마나 되는지 측정



Image segmentation using K-Means with various numbers of color clusters

* Matplotlib의 imread()함수를 이용하여 이미지 읽기

```
>>> from matplotlib.image import imread # you could also use `imageio.imread()`
>>> image = imread(os.path.join("images", "clustering", "ladybug.png"))
>>> image.shape
```

- (533, 800, 3) • 읽어들인 image는 3차원 array.
- 각 픽셀(좌표)에 RGB 강도를 담은 3D(0~1 사이값) 벡터가 있음.

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8).fit(X)
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
segmented_img = segmented_img.reshape(image.shape)
```

- 8개 미만의 클러스터 개수일 때, 무당벌레의 화려한 빨간색이 독자적인 클러스터를 만들지 못하고 주위 색에 합쳐짐.
- 무당벌레는 이미지의 나머지 부분보다 훨씬 작기 때문에 화려한 색을 가지고 있다 할지라도 k-평균이 무당벌레를 하나의 클러스터로 만들지 못함.

군집을 이용한 전처리

load_digits

- MNIST와 비슷한 데이터셋
- 0에서 9까지 숫자를 나타내는 8X8 (64 픽셀) 크기 흑백 이미지 1,797개를 담고 있음.
- 군집을 통한 전처리 시 모델 성능 증가 (96.6 → 98.2)

```
from sklearn.datasets import load_digits
```

```
X_digits, y_digits = load_digits(return_X_y=True)
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_digits, y_digits)
```

```
from sklearn.linear_model import LogisticRegression
```

```
log_reg = LogisticRegression(random_state=42)  
log_reg.fit(X_train, y_train)
```

```
>>> log_reg.score(X_test, y_test) 정확도  
0.9666666666666667
```

```
from sklearn.pipeline import Pipeline
```

```
pipeline = Pipeline([  
    ("kmeans", KMeans(n_clusters=50)),  
    ("log_reg", LogisticRegression()),  
)  
pipeline.fit(X_train, y_train)
```

```
>>> pipeline.score(X_test, y_test)  
0.9822222222222222
```

Logistic Regression

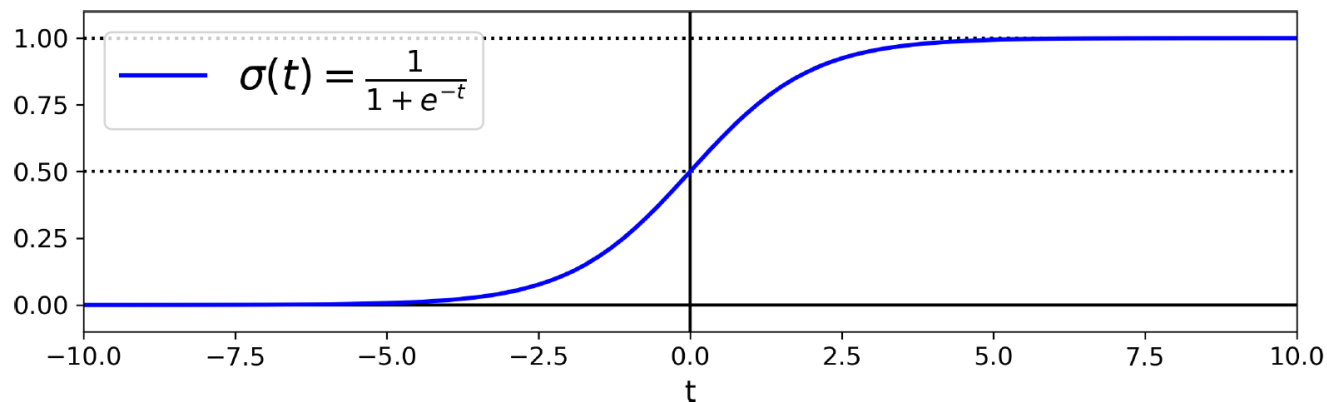
로지스틱 회귀

- 특정 클래스에 속할 확률을 추정하는데 사용됨.
- 예를 들면, 이 이메일이 스팸일 확률은? 추정 확률이 50%이상이면 해당 클래스에 속한다고 예측
- "훈련의 목표"는 양성샘플($y=1$)에 대해서는 높은 확률을 추정하고 음성샘플($y=0$)에 대해서는 낮은 확률을 추정하는 벡터 θ 를 찾는 것

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\mathbf{x}^T \theta)$$

- 로지스틱은 0과 1 사이의 값을 출력하는 sigmoid 함수 $\sigma(t) = \frac{1}{1 + \exp(-t)}$



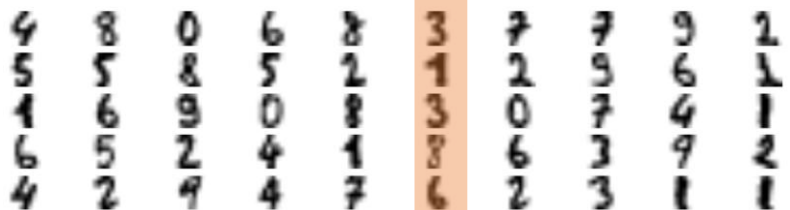
군집을 이용한 준지도 학습

준지도 학습

- 레이블이 없는 데이터가 많고, 레이블이 있는 데이터는 적을 때 사용.

각 클러스터를 대표하는 대표이미지 50개를
레이블하여 예측 성능 체크 (82.7 → 92.4)

```
k = 50
kmeans = KMeans(n_clusters=k)
X_digits_dist = kmeans.fit_transform(X_train)
representative_digit_idx = np.argmax(X_digits_dist, axis=0)
X_representative_digits = X_train[representative_digit_idx]
```



```
y_representative_digits = np.array([4, 8, 0, 6, 8, 3, ..., 7, 6, 2, 3, 1, 1])
```

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_representative_digits, y_representative_digits)
>>> log_reg.score(X_test, y_test)
0.9244444444444444
```

숫자 데이터셋에서 레이블된 50개 샘플(무작위로 고른)에 로지스틱 회귀 모델을 훈련.

```
n_labeled = 50
log_reg = LogisticRegression()
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])

>>> log_reg.score(X_test, y_test)
0.8266666666666667
```

무작위 샘플 대신 대표 샘플에 레이블을 할당하는 것이 더 좋은 방법!

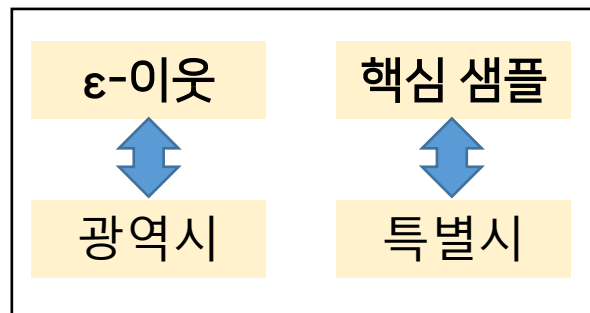
센트로이드와 가까운 샘플의 20%에만 레이블을 전파해보면 예측 성능은 더욱 증가!

전파된 레이블이 99% 실제데이터와 일치!

DBSCAN Clustering

DBSCAN

실습

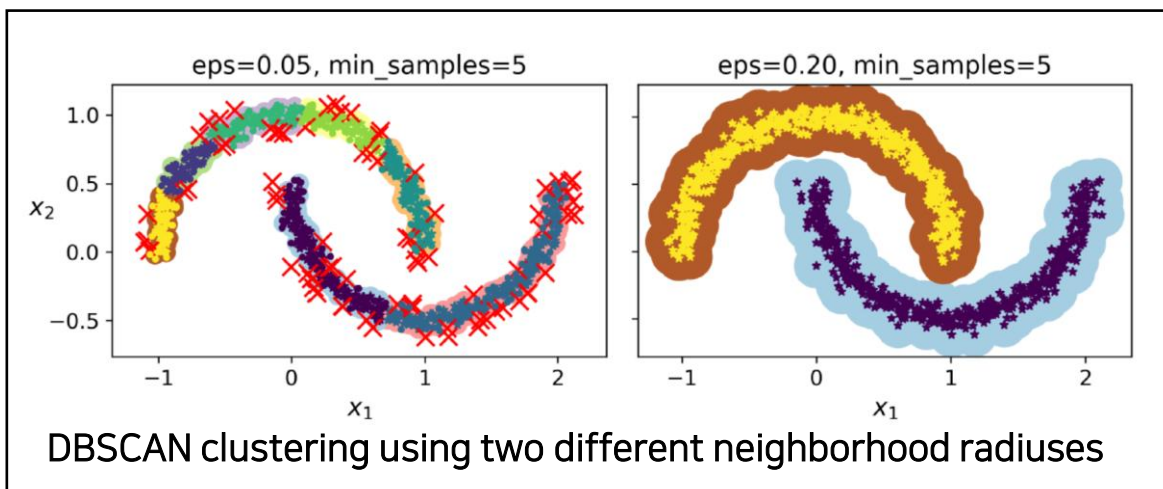


밀집된 연속적 지역을 클러스터로 정의하는 방식

- 모든 클러스터가 충분히 밀집되어 있고 밀집되지 않은 지역과 잘 구분될 때 좋은 성능을 나타냄

DBSCAN 작동방식

- 각 샘플에서 작은 거리 ϵ (입실론, eps) 내에 샘플이 몇 개 놓여 있는지 셈. 이 지역을 샘플의 ϵ -이웃(ϵ -neighborhood)이라고 부름.
- 자기 자신을 포함해 ϵ -이웃 내에 적어도 min-sample개 샘플이 있다면 이를 **핵심 샘플**(core instance)로 간주.
- 핵심 샘플의 이웃에 있는 모든 샘플은 동일한 클러스터에 속함. 이웃에는 다른 핵심 샘플이 포함될 수 있음. 따라서 핵심 샘플의 이웃의 이웃은 계속해서 하나의 클러스터를 형성**
- 핵심 샘플이 아니고 이웃도 아닌 샘플은 이상치로 판단.**



eps를 0.2로 증가해 샘플의 이웃 범위를 넓히면 더욱 완벽한 군집을 얻음

장점

- K-means와 달리 클러스터 수를 정할 필요 없음.
- 특이한 기하학적 모양을 갖는 군집도 찾을 수 있음.
- 이상치 검출도 가능.

단점

- 새로운 샘플에 대한 클러스터 예측이 불가능.
- 사용자가 필요한 예측기를 직접 선택해야 함.

ChemAI-DBSCAN 툴킷

<https://ai.chemdx.org/toolkits>

ChemAI Search... Seunghun Jang

Home Toolkits Machine Learning Applications Documentations Settings

Prediction of ZT (figure of merit) [more](#)

- Temperature (K)
- ZT (figure of merit)
- [Trained model](#)

Band Gap Prediction of Perovskites [more](#)

Band gap prediction of organic-inorganic perovskites

- Crystal structure (.cif)
- HSE band gap (eV)
- 0.901
- [Trained model](#)
- [Source dataset](#)

Band Gap Correction [more](#)

Prediction of G_0W_0 band gap from naive GGA band gap

- Crystal structure (.cif)
- GGA band gap (eV)
- G_0W_0 band gap (eV)
- 0.951
- [Trained model](#)
- [Source dataset](#)
- [Reference](#)

Prediction of Absorption Max [more](#)

Predicting absorption max of organic chromophores

- Chromophore structure (SMILES)
- Solvent structure (SMILES)
- Absorption max (nm)
- 0.902
- [Trained model](#)
- [Source dataset](#)

Data Clustering [more](#)

Grouping data points based on density of data

- Data points (.xlsx)
- Distance threshold (epsilon)
- Quantity threshold (MinPts)
- None
- Not available
- [Reference](#)

Data file: ChemAI_dbscan_toolkit_실습DB.xlsx [Browse](#)

Epsilon: 0.5

MinPts: 5 [Run](#)

Clustering result: [dataset_with_labels.xlsx](#)

Class Distribution

t-SNE Visualization [more](#)

Spectral Visualization [more](#)

Outlier Detection [more](#)

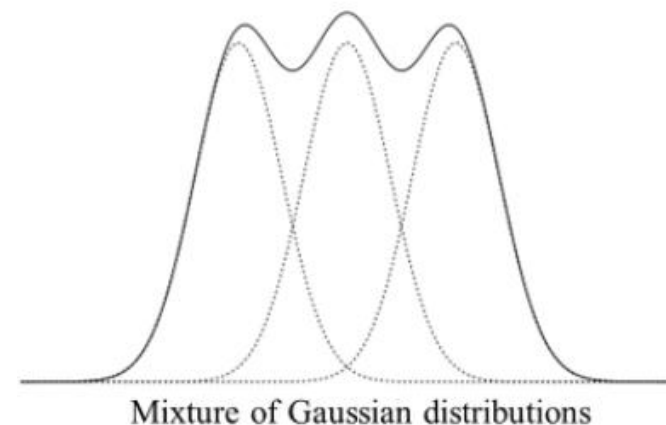
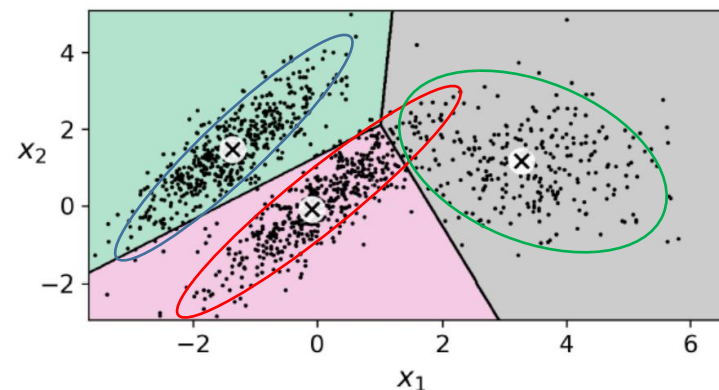
Detection of abnormal data based on data distribution

- Data points (.xlsx)
- Number of neighbors (NumNN)
- None
- Not available
- [Reference](#)

Gaussian mixture Model

가우시안 혼합 모델(Gaussian mixture model)

- 가우시안 혼합 모델은 샘플이 **파라미터가 알려지지 않은 여러 개의 혼합된 가우시안 분포에서 생성되었다고 가정하는 확률 모델**.
- **하나의 가우시안 분포에서 생성된 모든 샘플을 하나의 클러스터를 생성**. 일반적으로 이 클러스터는 타원.
- 각 클러스터는 오른쪽 그림처럼 **타원 모양, 크기, 밀집도, 방향**이 모두 다름.
- 샘플이 주어지면 가우시안 분포 중 하나에서 생성되었다는 것을 알지만 어떤 분포인지 또 이 분포의 파라미터는 무엇인지 알지 못함
- 이를 설명해주는 **평균, 분산, 가중치, 파라미터를 추정** 해야함.



가우시안 혼합 모델(Gaussian mixture model)

```
from sklearn.mixture import GaussianMixture
```

```
gm = GaussianMixture(n_components=3, n_init=10)
gm.fit(X)
```

```
>>> gm.weights_
```

```
array([0.20965228, 0.4000662 , 0.39028152])
```

```
>>> gm.means_
```

```
array([[ 3.39909717,  1.05933727],
       [-1.40763984,  1.42710194],
       [ 0.05135313,  0.07524095]])
```

```
>>> gm.covariances_
```

```
array([[[ 1.14807234, -0.03270354],
        [-0.03270354,  0.95496237]],
```

```
       [[ 0.63478101,  0.72969804],
        [ 0.72969804,  1.1609872 ]],
```

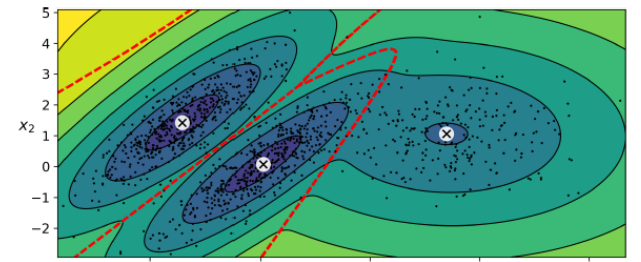
```
       [[ 0.68809572,  0.79608475],
        [ 0.79608475,  1.21234145]])])
```

- 데이터셋 X가 주어지면 가중치와 K개의 평균과 분산이 추정됨.
- K(n_components) 값을 설정해야함.
- 사이킷런의 GaussianMixture 클래스를 사용하여 간단하게 실행.
- Expectation-Maximization(EM) 알고리즘 사용됨. 클러스터 파라미터를 랜덤하게 초기화 하고 수렴할 때까지 EM 단계를 반복.
- 기대값 단계 : 샘플을 클러스터에 할당, 최대화 단계 : 클러스터를 업데이트함.
- 실제 이 데이터를 생성하기 위해서 사용한 가중치는 0.2, 0.4, 0.4
- Gaussian mixture model 이 찾은 것과 매우 흡사.
- K-means 처럼 나쁜 결과로 수렴할 수도 있음. 따라서 여러 번 실행하여 가장 좋은 결과를 선택해야 함. 이 때문에 n_init를 10으로 설정. (n_init의 기본값은 1)
- 알고리즘 수렴여부와 반복횟수 확인 가능.

```
>>> gm.converged_
True
```

```
>>> gm.n_iter_
3
```

가우시안 혼합 모델(Gaussian mixture model)



클러스터 평균, 결정경계(파선), 밀도 등고선

```
>>> gm.predict(X)
array([2, 2, 1, ..., 0, 0, 0])

>>> gm.predict_proba(X)
array([[2.32389467e-02, 6.77397850e-07, 9.76760376e-01],
       [1.64685609e-02, 6.75361303e-04, 9.82856078e-01],
       [2.01535333e-06, 9.99923053e-01, 7.49319577e-05],
       ...,
       [9.99999571e-01, 2.13946075e-26, 4.28788333e-07],
       [1.00000000e+00, 1.46454409e-41, 5.12459171e-16],
       [1.00000000e+00, 8.02006365e-41, 2.27626238e-15]])
```

- GM 모델은 새로운 샘플을 가장 비슷한 클러스터에 손쉽게 할당.
- `predict()` : 어느 클러스터에 할당되었는지 확인
- `predict_proba()` : 각 클러스터(3개)에 속할 확률 예측.

```
>>> gm.score_samples(X)
array([-2.60782346, -3.57106041, -3.33003479, ..., -3.51352783,
       -4.39802535, -3.80743859])
```

- `score_sample()` 메서드는 그 위치에서의 **확률밀도** 값을 예측.

```
>>> X_new, y_new = gm.sample(6)
>>> X_new
array([[ 2.95400315,  2.63680992],
       [-1.16654575,  1.62792705],
       [-1.39477712, -1.48511338],
       [ 0.27221525,  0.690366  ],
       [ 0.54095936,  0.48591934],
       [ 0.38064009, -0.56240465]])
```

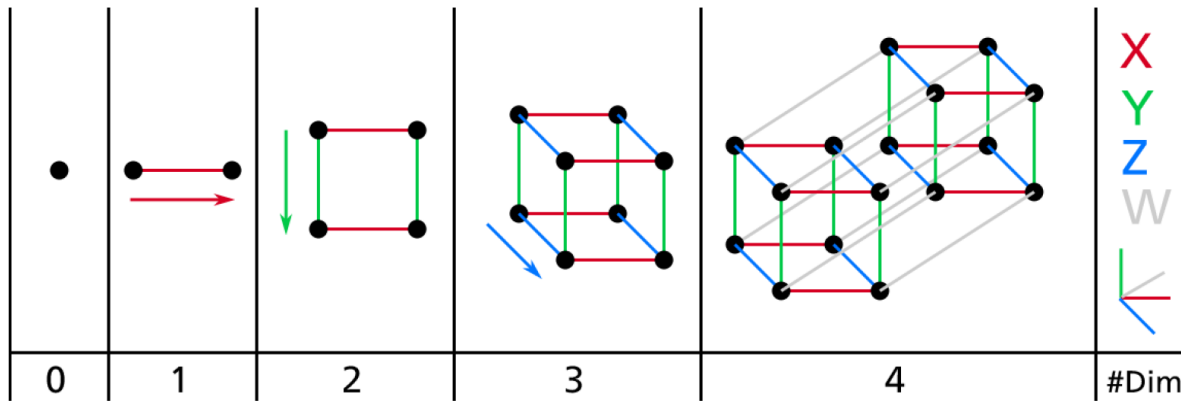
```
>>> y_new
array([0, 1, 2, 2, 2, 2])
```

- GM 모델은 생성 모델이기도 함.
- 오른쪽 처럼 `gm.sample(6)` 이라고 쓰면, 새로운 X, y 값들이 생성됨.

차원축소법 PCA, LLE, 오토인코더

Principal Component Analysis (PCA)

차원의 저주 (The Curse of Dimensionality)



Point, segment, square, cube, and tesseract (0D to 4D hypercubes)

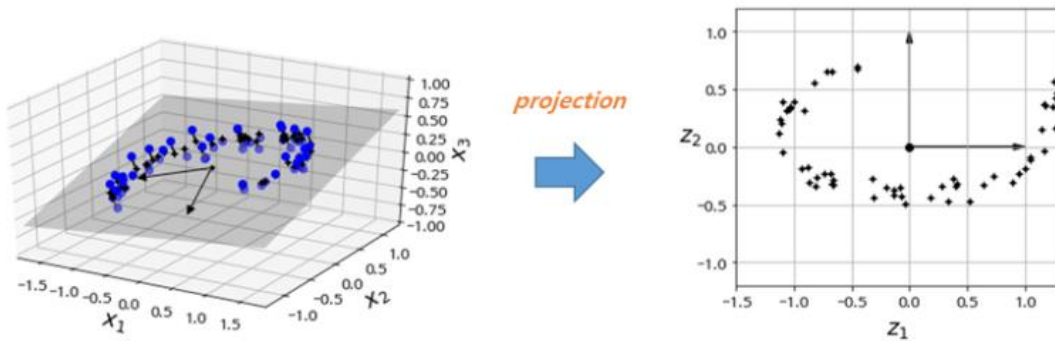
- 우리들은 3차원 공간에 살고 있음.
- 고차원 공간을 상상하기 쉽지 않음.
- 단위 면적에서 임의의 두 점을 선택하면 **두 점 사이 거리는 평균적으로 0.52**
- 단위 공간에서 임의의 두 점을 선택하면 두 점 사이 거리는 평균적으로 0.66
- 1000,000 차원의 초 입방체에서 두 점을 무작위로 선택하면 **평균거리는 408.25**
- **고차원은 많은 공간을 가지게 되므로**, 고차원 데이터셋은 공간상에서 매우 희박한 데이터셋일 위험성을 가짐.
- 훈련 샘플 간의 **거리도 멀다는 의미가 되며 이 경우 매우 많은 extrapolation이 필요하므로**, 저차원에 비해 예측이 훨씬 불안정!!
- 간단히 말하면, 데이터셋의 차원(Feature)이 커지면 과대적합 위험이 커짐.
- 해결방안: **훈련세트의 크기를 키우는 것**, 그러나 **차원수가 커짐에 따라 필요한 훈련 샘플수는 기하급수적으로 늘어나므로** 훈련데이터 개수를 늘리는 것은 대부분의 경우 **현실적으로 불가능**.

차원 축소 방법 필요

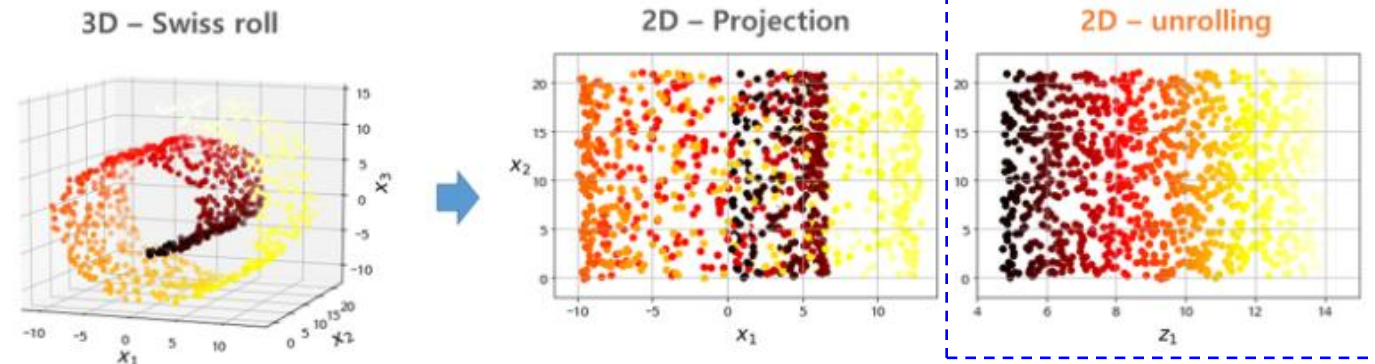


투영 (Projection)

- 대부분의 실제 훈련 샘플들은 모든 차원에 걸쳐 균일하게 퍼져 있지 않음
- 고차원 공간 안의 저차원 부분 공간(subspace)에 가까이 놓여 있음
= 고차원 데이터의 특성 중 **일부특성으로 데이터 표현**이 가능
- 3차원 공간 데이터셋 → 2차원 평면 데이터셋



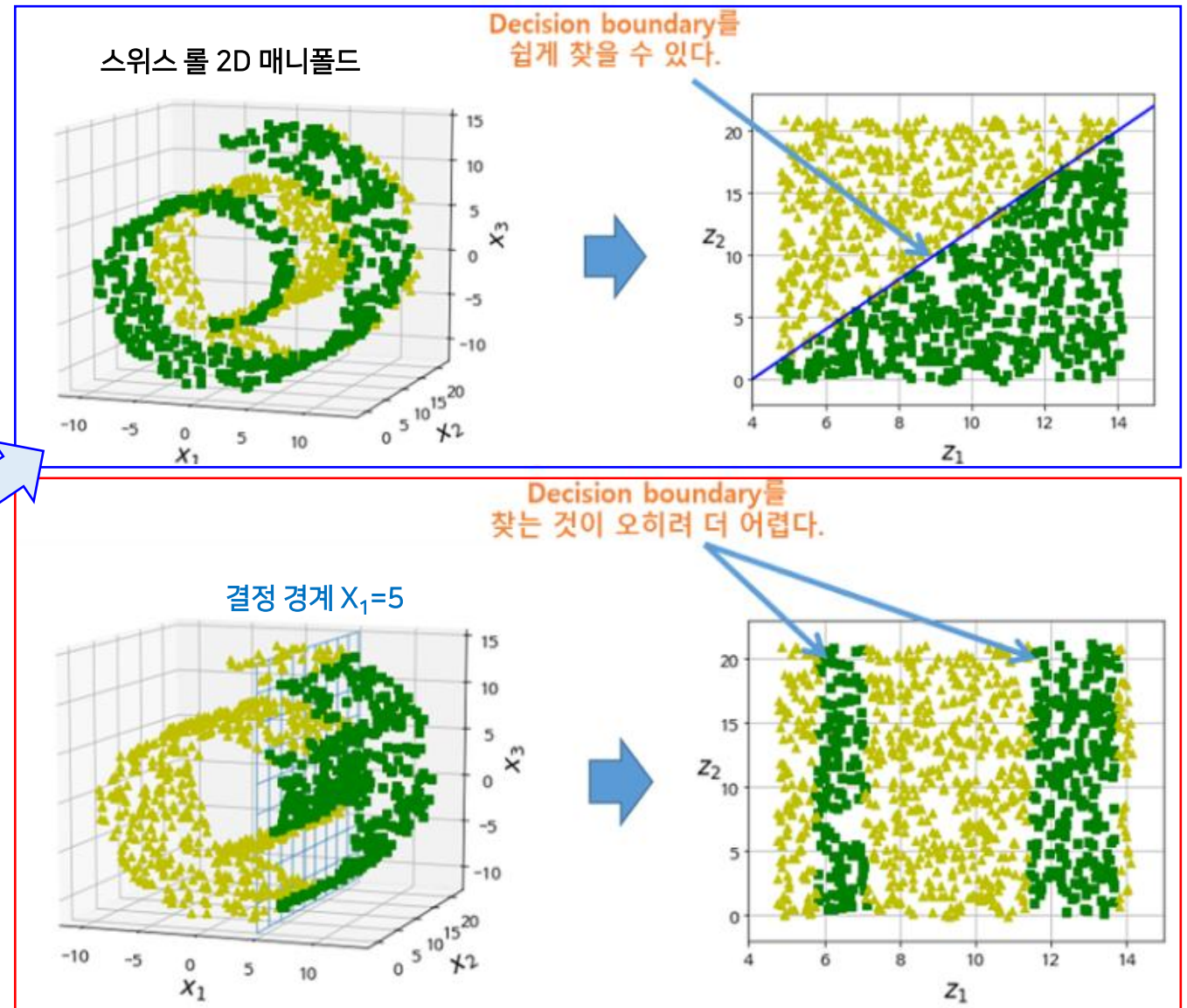
- 그러나 **차원 축소**에 있어 투영이 언제나 최선은 아님.
- 스위스 롤(부분 공간이 뒤틀려 있거나 휘어 있음, 매니폴드)
- 2D(x_3 축 값을 버림, 평면 투영) - 데이터 뭉개짐
- 우리가 원하는 것은 스위스 롤을 펼쳐서 2D 데이터셋을 얻는것



매니폴드 예시

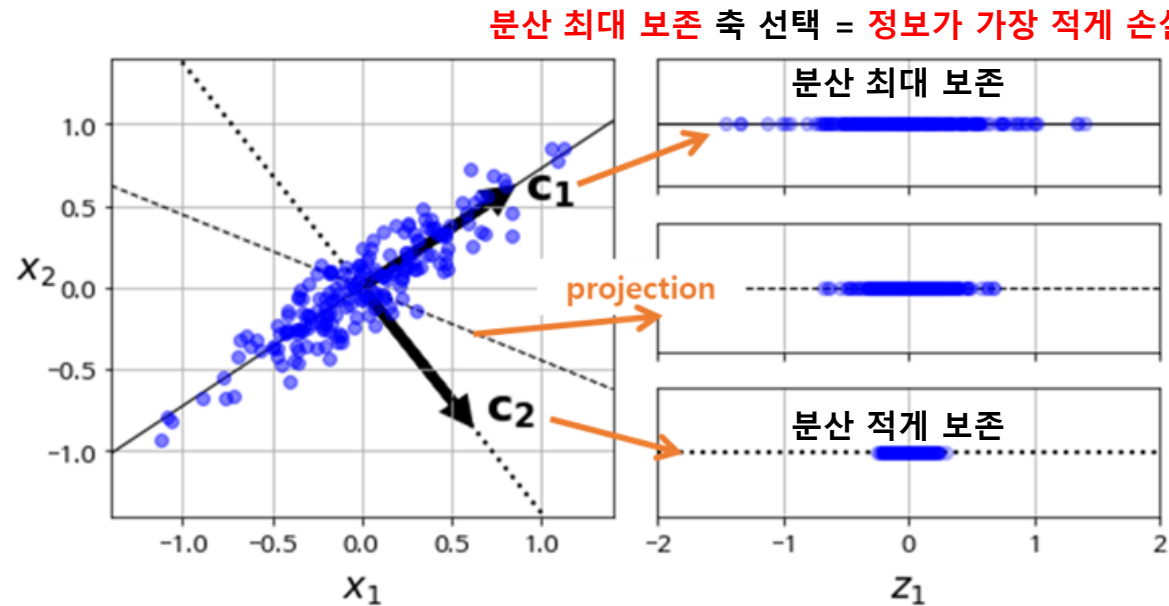
매니폴드 학습(manifold learning)

- 스위스 롤은 2D 매니폴드의 한 예
- "2D 매니폴드"는 "고차원 공간에서 휘어지거나 뒤틀린 2D 모양"
- 일반적으로 d 차원 매니폴드는 국부적으로 d 차원 초평면으로 보일 수 있는 n 차원 공간의 일부 ($d < n$)
- 스위스 롤 ($d=2, n=3$)
- 많은 차원 축소 알고리즘들이 훈련 샘플이 놓여있는 매니폴드를 모델링 하는 식으로 작동함.
- 대부분의 실제 고차원 데이터셋이 더 낮은 저차원 매니폴드에 가깝게 놓여 있다는 매니폴드 가정(manifold assumption)에 근거하여 차원 축소가 수행됨.
- 그러나 이러한 가정이 항상 유효한 것은 아님.
- 펼쳐진 매니폴드에서 결정 경계가 더 복잡해졌음.
- 데이터셋을 잘 파악하는 것이 필요.



Principal Component Analysis (주성분 분석)

- 가장 대표적인 차원 축소 알고리즘
 - 데이터에 가장 가까운 **초평면을 정의**한 다음, 데이터를 **평면에 투영**시킨다.
= 원본데이터셋과 투영된 것 사이의 평균 제곱 거리를 최소화 하는 축
 - 주요성분 찾기, 분산이 최대 보존되는 축 찾기

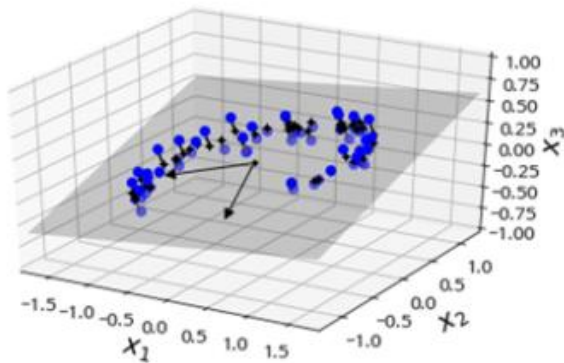


1. 분산이 최대인 축을 찾고 (첫번째 축)
2. 첫번째 축에 직교, 남은 분산을 최대한 보존하는 두번째 축을 찾는다.
3. 데이터 셋의 차원만큼 i 번째 축 ($=i$ 번째 주성분(PC))을 찾는다.

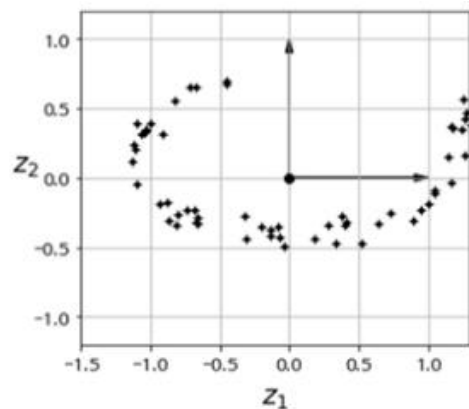
주성분 추출 → d차원으로 투영

$$X_{d\text{-proj}} = XW_d$$

- $X_{d\text{-proj}}$ (60,2) - d 차원으로 축소된 데이터셋
- X (60,3) - 원래 데이터셋
- W_d (3,2) - V의 첫 d열로 구성된 행렬



projection



분산을 최대한 유지하는 초평면 위에
원래 데이터셋을 투영 시킨 결과

결과를 보면 2D 투영결과는
원본 3D 데이터셋과 비슷해보임.

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components = 2)  
X2D = pca.fit_transform(X)
```



PCA 학습이 끝나면 component_ 속성에 주성분이 담겨 있음.

→ 첫번째 주성분의 단위벡터: `pca.components_.T[:,0]`

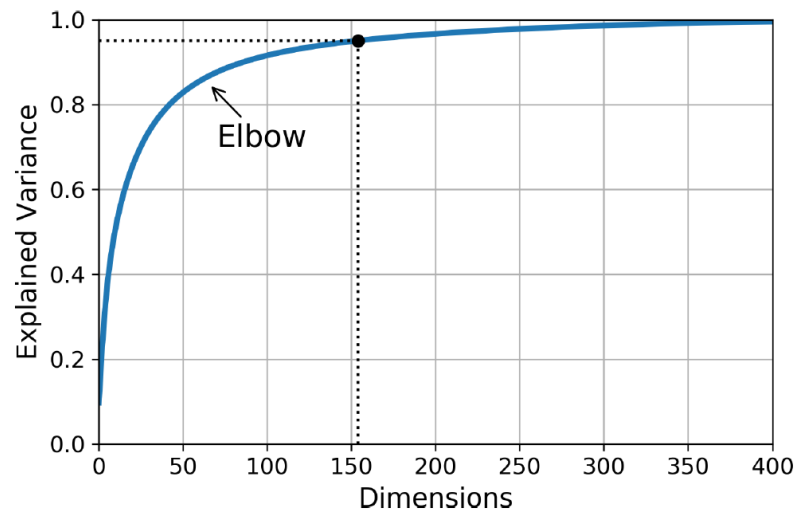
주성분 분석-적절한 차원수 선택

- 축소할 차원 수를 임의로 정하기 보다 충분한 분산(예, 95%)이 될 때까지 더해야 할 차원 수를 선택하는 것이 간단
- 시각화를 염두한다면 2차원 또는 3차원으로 줄여야 함

```
pca = PCA(n_components=0.95)  
X_reduced = pca.fit_transform(X_train)
```

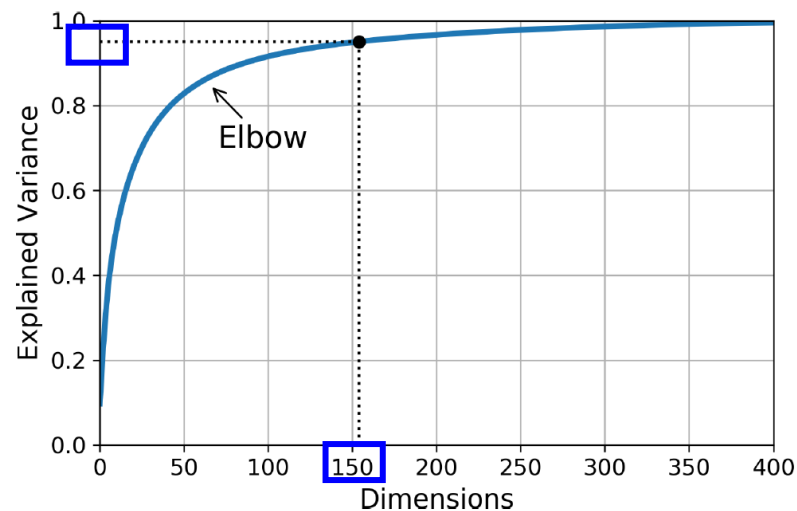
보존하려는 분산의 비율을
0.0-1.0 사이로 설정

- 또 다른 방법은 설명된 분산을 차원 수에 대한 함수로 그리는 것.
- 차원을 100으로 축소해도 설명된 분산을 크게 손해보지 않음.



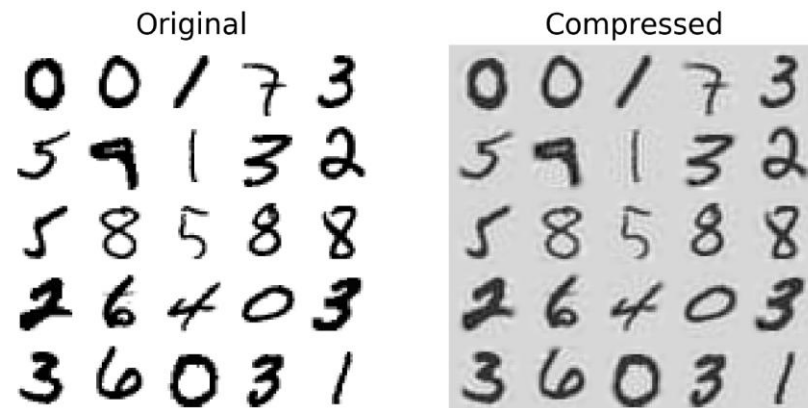
주성분 분석-압축

- 차원 축소 후에 훈련 세트의 크기가 줄어듦.
- MNIST 데이터셋에 분산의 95% 유지하도록 PCA 적용.
- 원래 784개 특성이 아닌 **150개** 정도만 가져도 **분산의 95% 유지**
- 데이터셋의 크기는 원본 크기 대비 20% 미만 (상당한 압축률)
- 압축된 데이터셋을 적용시 SVM과 같은 분류 알고리즘의 속도 향상.



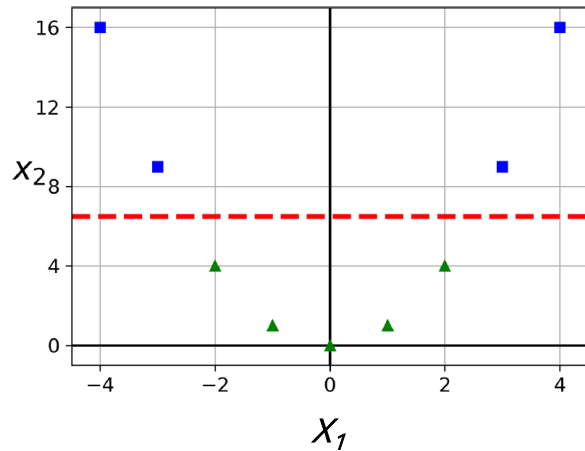
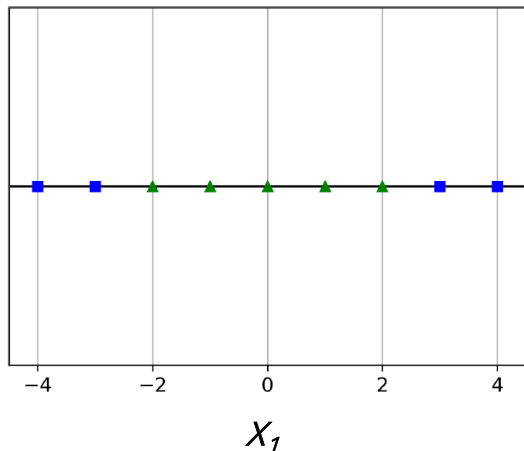
- PCA 투영 변환을 반대로 적용하여 784 차원으로 돌릴 수 있음. (복원)
- 5% 정도의 분산을 잃어버렸으므로 원본으로 돌아가진 않음.
- 그러나 원본 데이터와 매우 흡사
- 원본과 재구성된 데이터 사이의 평균 제곱 거리를 **재구성 오차** 라고 함

```
pca = PCA(n_components = 154)
X_reduced = pca.fit_transform(X_train)
X_recovered = pca.inverse_transform(X_reduced)
```

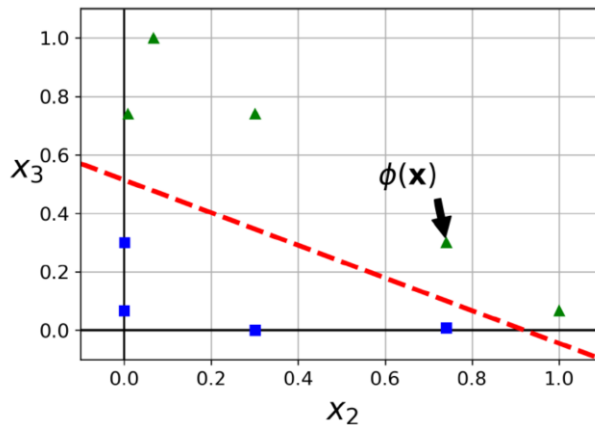
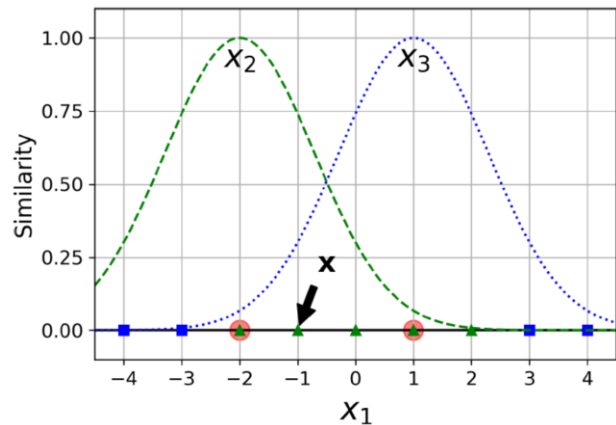


Kernel Trick

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
])
poly_kernel_svm_clf.fit(X, y)
```



- 왼쪽 그래프는 하나의 특성 x_1 을 가진 데이터셋.
- 그림에서 보는 바와 같이 구분이 불가능.
- $x_2 = (x_1)^2$ 을 추가하여 2차원 데이터셋을 만들면 완벽하게 선형적으로 구분할 수 있음. → 사이킷런의 `PolynomialFeature(degree=n)`을 사용.
- 이처럼 실제로는 특성을 추가하지 않으면서 수학적 기교를 통해서 다항식 특성을 많이 추가한 것과 같은 결과를 얻는 것을 **커널 트릭** 이라고 함. (SVC polynomial kernel)



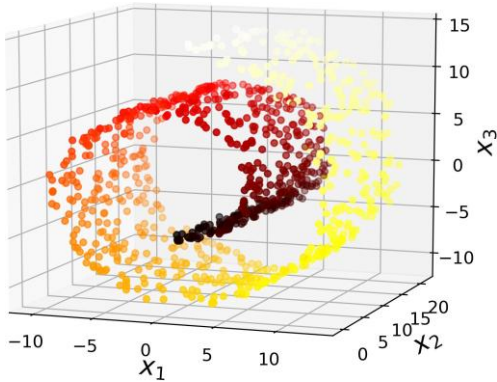
- -2, 1 위에 x_2, x_3 랜드마크를 세우고(왼쪽 그래프), 각 점마다 Gaussian radial basis function을 계산하면 오른쪽 그래프처럼 2차원의 새로운 변환된 데이터셋을 보여짐.
- Gaussian RBF 커널을 사용하면 선형적으로 구분이 가능함.

$$\phi_\gamma(\mathbf{x}, \ell) = \exp(-\gamma \|\mathbf{x} - \ell\|^2) \quad \gamma = 0.3$$

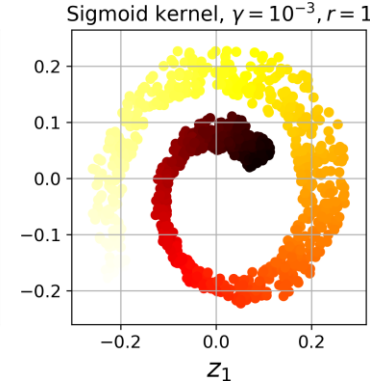
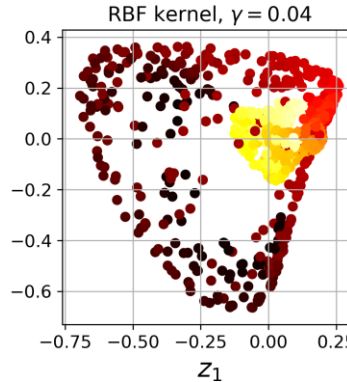
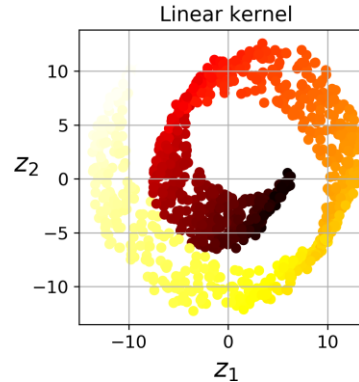
Kernel PCA

원본 공간에서는 복잡한 비선형 결정 경계 → 고차원 공간에서의 선형 결정 경계

앞서 언급한 수학적 기법을 통한 커널 트릭을 PCA에 적용
→ 차원축소를 위한 복잡한 비선형 투영을 수행. (Kernel PCA)



Swiss roll dataset



Swiss roll reduced to 2D using kPCA with various kernels

Kernel PCA

- 비지도 학습 → 명확한 성능측정 기준 없음.
- 주로 지도학습의 전처리 단계로 사용됨.
 - 커널 및 하이퍼파라미터(HP) 선택 → 로지스틱 회귀 적용
 - GridSearchCV 사용하여 가장 좋은 커널 및 가장 좋은 HP 선택.

Kernel PCA-하이퍼파라미터 튜닝

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

clf = Pipeline([
    ("kpca", KernelPCA(n_components=2)),
    ("log_reg", LogisticRegression())
])

param_grid = [{
    "kpca__gamma": np.linspace(0.03, 0.05, 10),
    "kpca__kernel": ["rbf", "sigmoid"]
}]

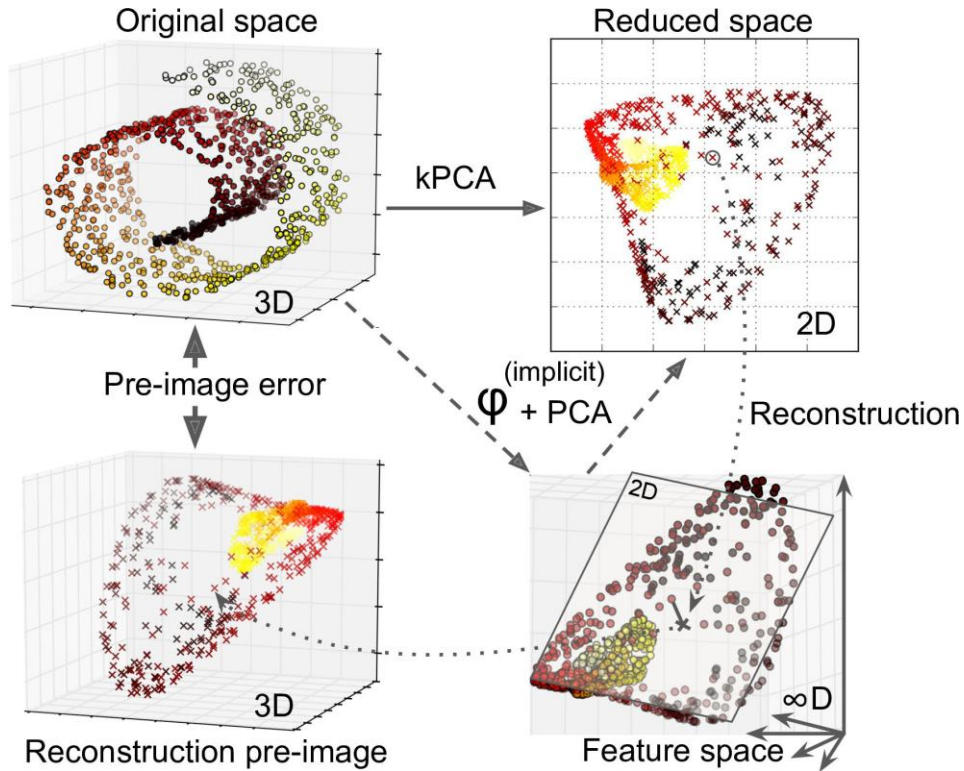
grid_search = GridSearchCV(clf, param_grid, cv=3)
grid_search.fit(X, y)
```

커널 PCA와 로지스틱 회귀를
하이퍼 파라미터를 바꿔가면서 성능을 체크하고
최적의 성능을 나타내는 하이퍼 파라미터를 찾음



```
>>> print(grid_search.best_params_)
{'kpca__gamma': 0.043333333333333335, 'kpca__kernel': 'rbf'}
```

Kernel PCA



Kernel PCA and the reconstruction pre-image error

Kernel PCA

■ 완전한 비지도 학습(방법 제안)

가장 **낮은 재구성 오차**를 만드는 커널과 HP 선택하는 방식이 있음. → 그러나 재구성은 선형 PCA 만큼 쉽지 않음.

1. 특성 맵 Φ 을 사용하여 훈련 세트를 무한 차원의 특성 공간에 매핑 → 변환된 데이터셋을 선형 PCA를 사용하여 2D투영 (커널트릭 이용)
2. 축소된 공간 샘플에 대한 PCA를 역전시키면, 재구성된 데이터포인트는 원본공간이 아닌 특성공간에 위치
3. 이 특성 공간은 무한 차원이므로 재구성된 포인트를 계산할 수 없고, 재구성에 따른 실제 에러를 계산할 수 없음.
4. 재구성된 포인트에 가깝게 매핑된 원본공간의 포인트를 찾을 수 있음

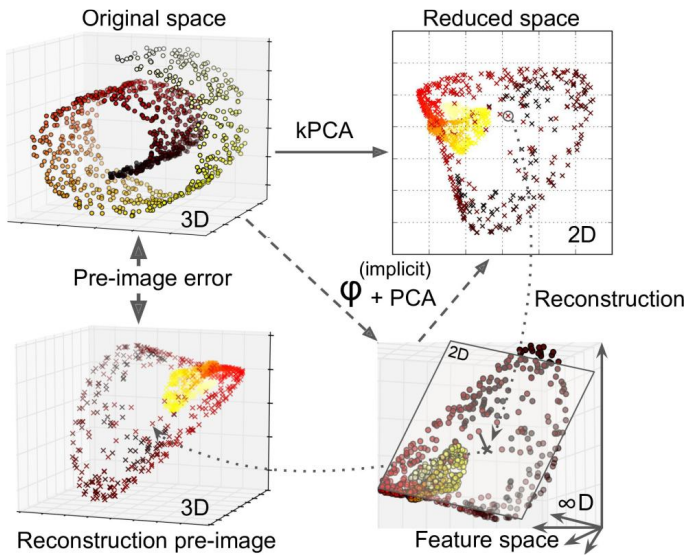
Kernel PCA

```
rbf_pca = KernelPCA(n_components = 2, kernel="rbf", gamma=0.0433,  
                    fit_inverse_transform=True)  
X_reduced = rbf_pca.fit_transform(X)  
X_preimage = rbf_pca.inverse_transform(X_reduced)
```



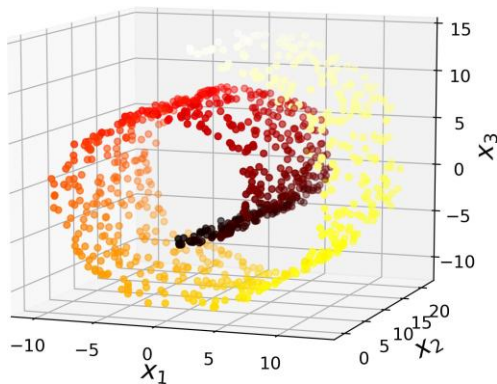
```
>>> from sklearn.metrics import mean_squared_error  
>>> mean_squared_error(X, X_preimage)  
32.786308795766132
```

원본 데이터셋과 재구성된 pre-image 간의
MSE 계산

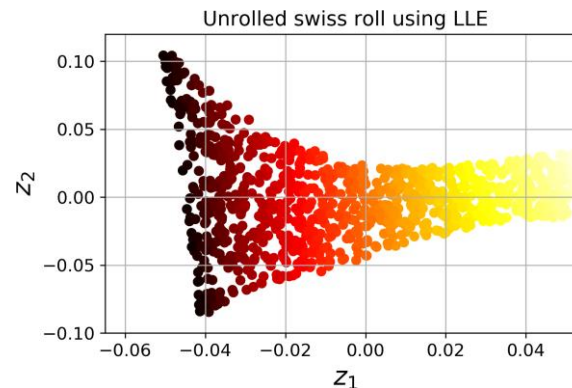


Locally Linear Embedding (LLE)

Locally Linear Embedding (LLE)



Swiss roll dataset

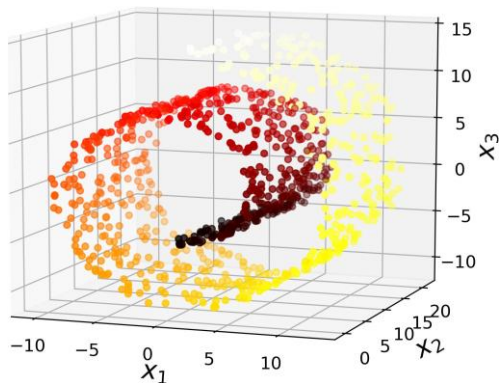


Unrolled Swiss roll using LLE

LLE

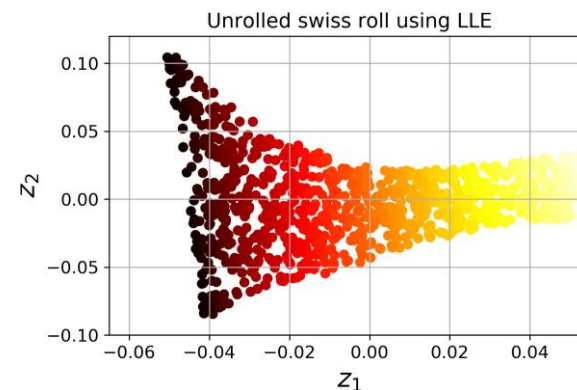
- 또 다른 강력한 비선형 차원 축소 방법.
- 투영에 의존하지 않는 매니폴드 학습
- 각 훈련샘플이 가장 가까운 이웃에 얼마나 선형적으로 연관되어 있는지 측정
- 그런다음, 국부적인 관계가 가장 잘 보존되는 훈련세터의 저차원 표현을 찾음.
- 이 방법은 잡음이 너무 많지 않은 경우 꼬인 매니폴드를 펼치는데 잘 작동함.

Locally Linear Embedding (LLE)



Swiss roll dataset

```
from sklearn.manifold import LocallyLinearEmbedding  
  
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10)  
X_reduced = lle.fit_transform(X)
```



Unrolled Swiss roll using LLE



- 스위스 롤이 완전히 펼쳐졌음.
- 지역적으로 샘플간 거리가 잘 보존되어 있음.
- 그러나 크게 보면 샘플간 거리가 잘 유지되어 있지 않음.
- 스위스 롤의 오른쪽은 압축, 왼쪽은 확장되어 있음.
- 그러나 매니폴드를 모델링 하는데 잘 작동!!

Locally Linear Embedding (LLE)

$$\begin{matrix} & & & m \\ & & & \\ & & & \\ n & & & \\ \text{(데이터 개수)} & & & \end{matrix} = W_{ij}$$

Equation 8-4. LLE step 1: linearly modeling local relationships

$$\begin{aligned} \widehat{W} = \underset{W}{\operatorname{argmin}} \quad & \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2 \quad \text{n차원 공간} \quad \text{n_neighbors} = j \\ \text{subject to} \quad & \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ c.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^m w_{i,j} = 1 & \text{for } i = 1, 2, \dots, m \end{cases} \quad \text{가중치 정규화 (합은 1)} \end{aligned}$$

최적화된 가중치 전이

Equation 8-5. LLE step 2: reducing dimensionality while preserving relationships

$$\widehat{Z} = \underset{Z}{\operatorname{argmin}} \quad \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \widehat{w}_{i,j} \mathbf{z}^{(j)} \right)^2 \quad \text{d차원 공간}$$

훈련샘플 $x^{(i)}$ 에 대해 가장 가까운 m 개의 샘플을 찾는다.

선형함수로 $x^{(i)}$ 를 재구성 $\rightarrow \sum_{j=1}^m w_{i,j} x^{(j)}$

$x^{(i)}$ 와 $\sum_{j=1}^m w_{i,j} x^{(j)}$ 사이의 제곱거리가 최소로 되는 $w_{i,j}$ 를 찾는다.

$x^{(i)}$ 가 $x^{(i)}$ 의 가장 가까운 이웃 중 하나가 아니면 $w_{i,j}=0$.

$w_{i,j}$ 을 모두 담은 가중치 행렬은 은 훈련 샘플 사이에 있는 지역 선형 관계를 담고 있음

이러한 관계가 잘 보존되도록 훈련샘플을 저 차원 공간으로 mapping수행.

$z^{(i)}$ 는 d차원 공간에서 $x^{(i)}$ 의 상(image)이라고 할때

$z^{(i)}$ 와 $\sum_{j=1}^m \widehat{w}_{i,j} z^{(j)}$ 사이의 거리가 최소화 되어야 함.

첫번째 단계와 비슷해 보이지만, 2번째 단계에서는 **가중치는 고정하고 저차원 공간에서 샘플 이미지의 최적 위치를 바꾸며** 찾아감.

Other Dimensionality Reduction Techniques

랜덤 투영

- 랜덤한 선형 투영을 사용해 데이터를 저차원 공간으로 투영

다차원 스케일링 (MDS)

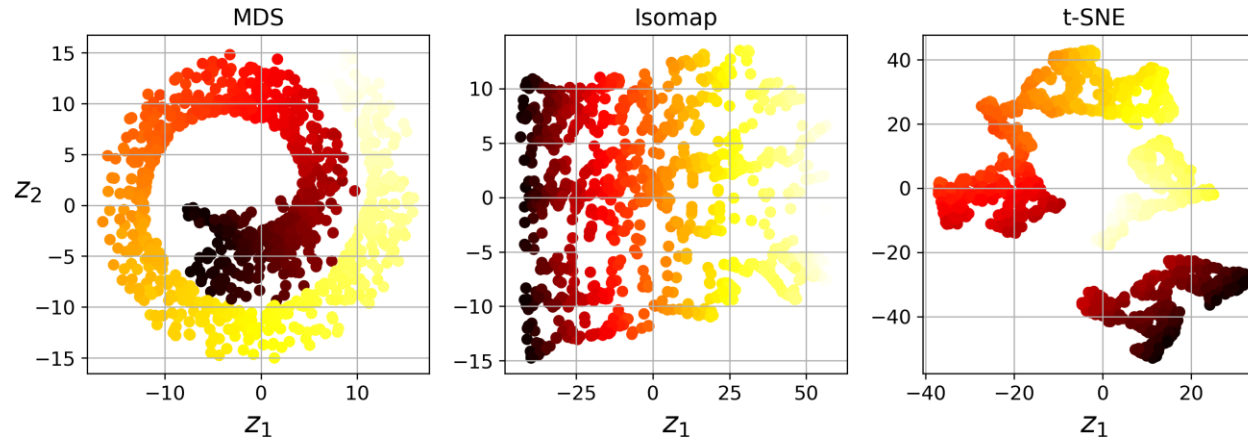
- 샘플간의 거리를 보존하면서 차원을 축소

Isomap

- 각 샘플을 가장 가까운 이웃과 연결하는 식으로 그래프를 만듦, 샘플간의 지오데식 거리를 유지하면서 차원을 축소.

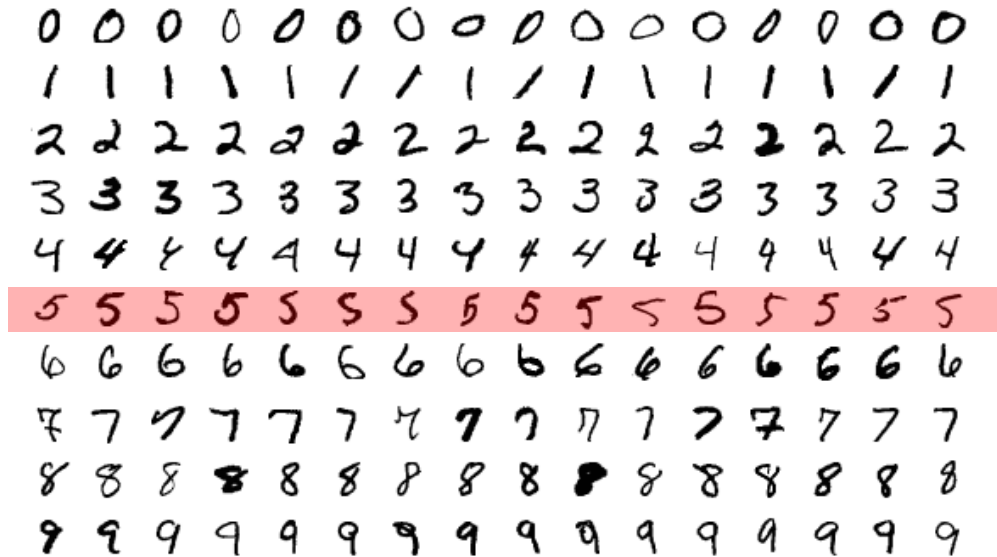
t-SNE

- 비슷한 샘플은 가까이, 비슷하지 않은 샘플은 멀리 떨어지도록 하면서 차원을 축소. 주로 시각화에 사용. 고차원 공간에 있는 샘플의 군집을 시각화 할때 사용. (MNIST 데이터셋(784 차원)을 2D로 시각화할 때)

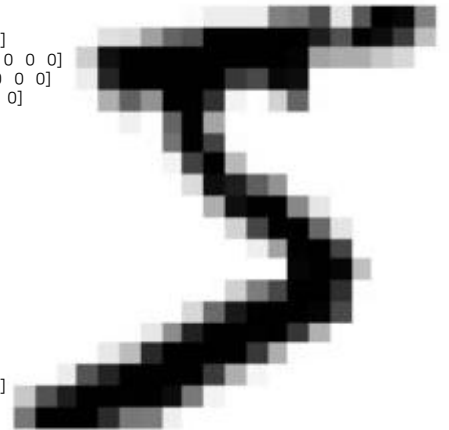


MNIST Dataset

- 미국 고등학생들과 인구조사국 직원들이 **손으로 쓴 작은 숫자** 이미지.



위 그림과 같이 0에서 9까지 10가지로 분류될 수 있는 손글씨 숫자 이미지 70,000개로 구성.

[illegible]

각 이미지는 28×28 픽셀로 구성.
각 픽셀은 위와 같이 0~255 사이의 숫자 행렬로 표현.

MNIST Dataset

dict_keys(['data', 'target', 'frame', 'categories', 'feature_names', 'target_names', 'DESCR', 'details', 'url'])

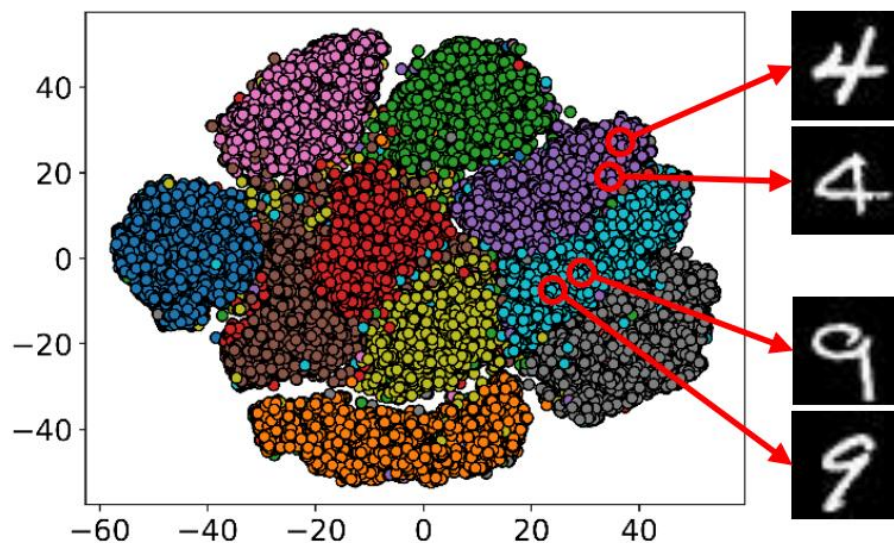
```
{'data': array([[0., 0., 0., ..., 0., 0., 0.],  
               [0., 0., 0., ..., 0., 0., 0.],  
               [0., 0., 0., ..., 0., 0., 0.],  
               ...,  
               [0., 0., 0., ..., 0., 0., 0.],  
               [0., 0., 0., ..., 0., 0., 0.],  
               [0., 0., 0., ..., 0., 0., 0.])),  
'target': array(['5', '0', '4', ..., '4', '5', '6'], dtype=object),  
'frame': None,  
'categories': {},  
'feature_names': ['pixel1', 'pixel2', 'pixel3', 'pixel4', ..., 'pixel784'],  
'target_names': ['class'],  
'DESCR': """Author: Yann LeCun,""" Source: [MNIST Website](http://yan~  
'details': {'id': '554', 'name': 'mnist_784', 'version': '1', ...}  
'url': 'https://www.openml.org/d/554'}
```



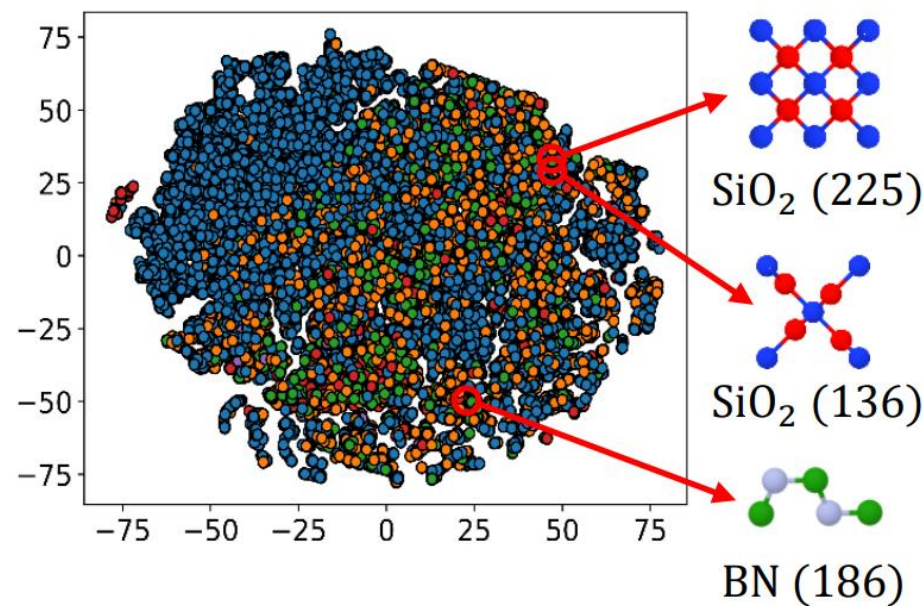
5	0	4	1	9	2	1	3	1	4
3	5	3	6	1	7	2	8	6	9
4	0	9	1	1	2	4	3	2	7
3	8	6	9	0	5	6	0	7	6
1	8	1	9	3	9	8	5	9	3
3	0	7	4	9	8	0	9	4	1
4	4	6	0	4	5	6	1	0	0
1	7	1	6	3	0	2	1	1	7
8	0	2	6	7	8	3	9	0	4
6	7	4	6	8	0	7	8	3	1

연구에 사용된 차원 축소(t-SNE) 예시

(a) MNIST dataset



(b) MP dataset



Nonlinearity Encoding for Extrapolation of Neural Networks

Gyoung S. Na, Chanyoung Park

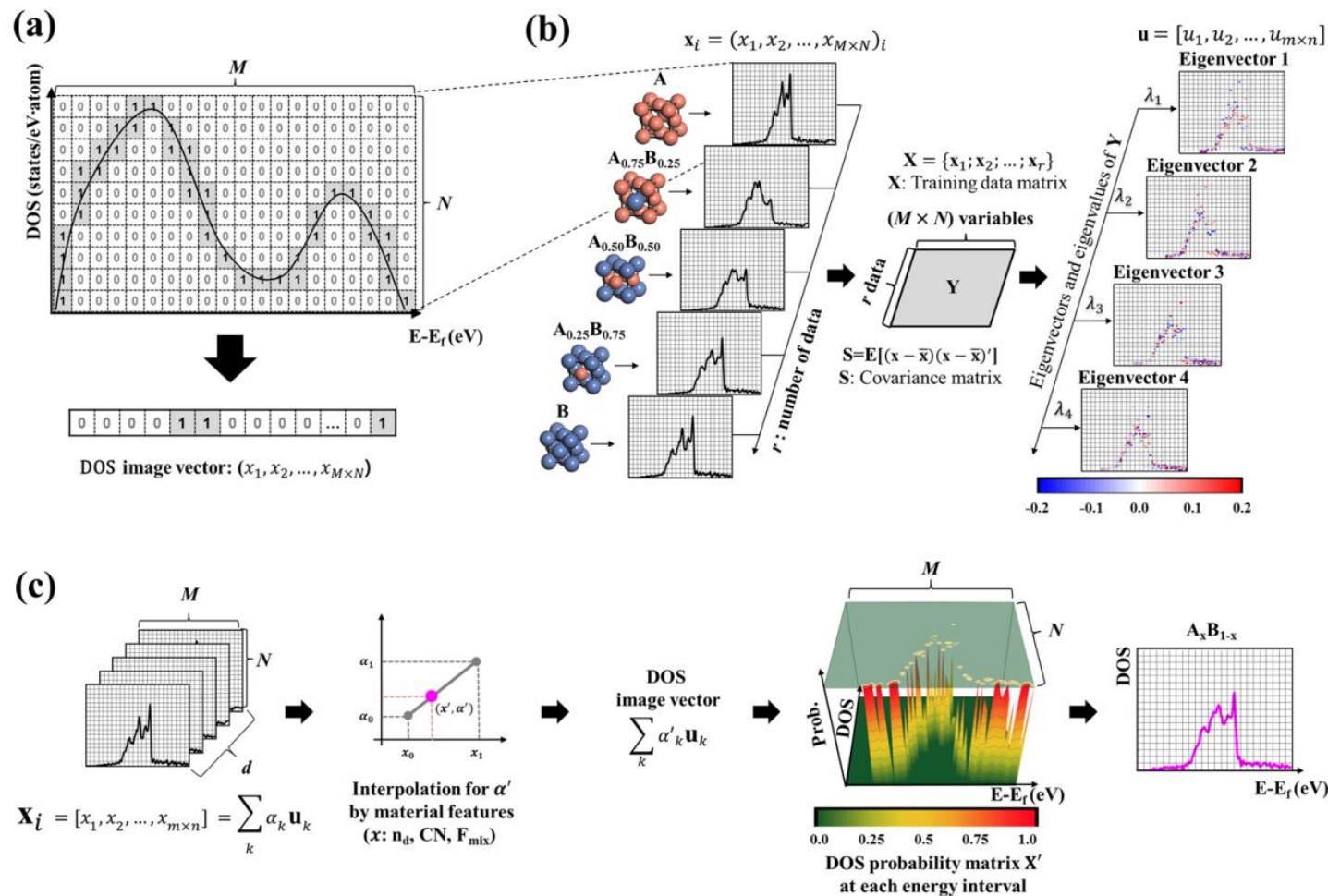
KDD 2022 - ACM SIGKDD Conference on Knowledge Discovery and Data Mining

연구에 사용된 차원 축소(PCA) 예시

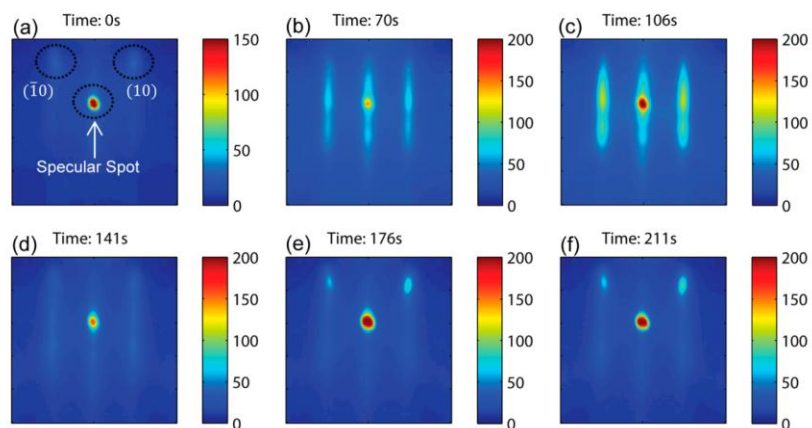
Pattern Learning Electronic Density of States

Byung Chul Yeo, Donghun Kim, Chansoo Kim & Sang Soo Han

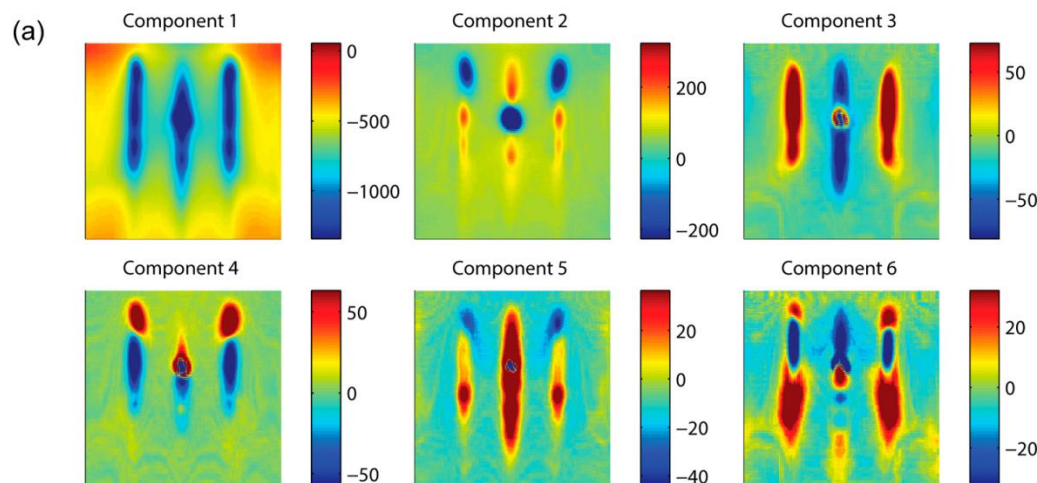
Scientific Reports 9, 5879 (2019)



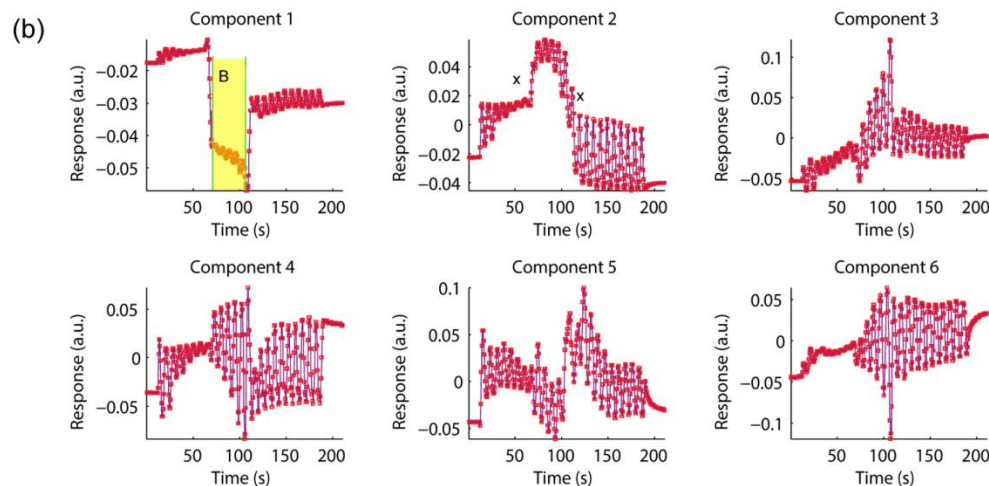
연구에 사용된 차원 축소(PCA) 예시



Frames from captured RHEED data of $\text{La}_{3/8}\text{Ca}_{5/8}\text{MnO}_3$ growing on (001) SrTiO_3 substrate at selected times.



eigenvectors
and
eigenvalues



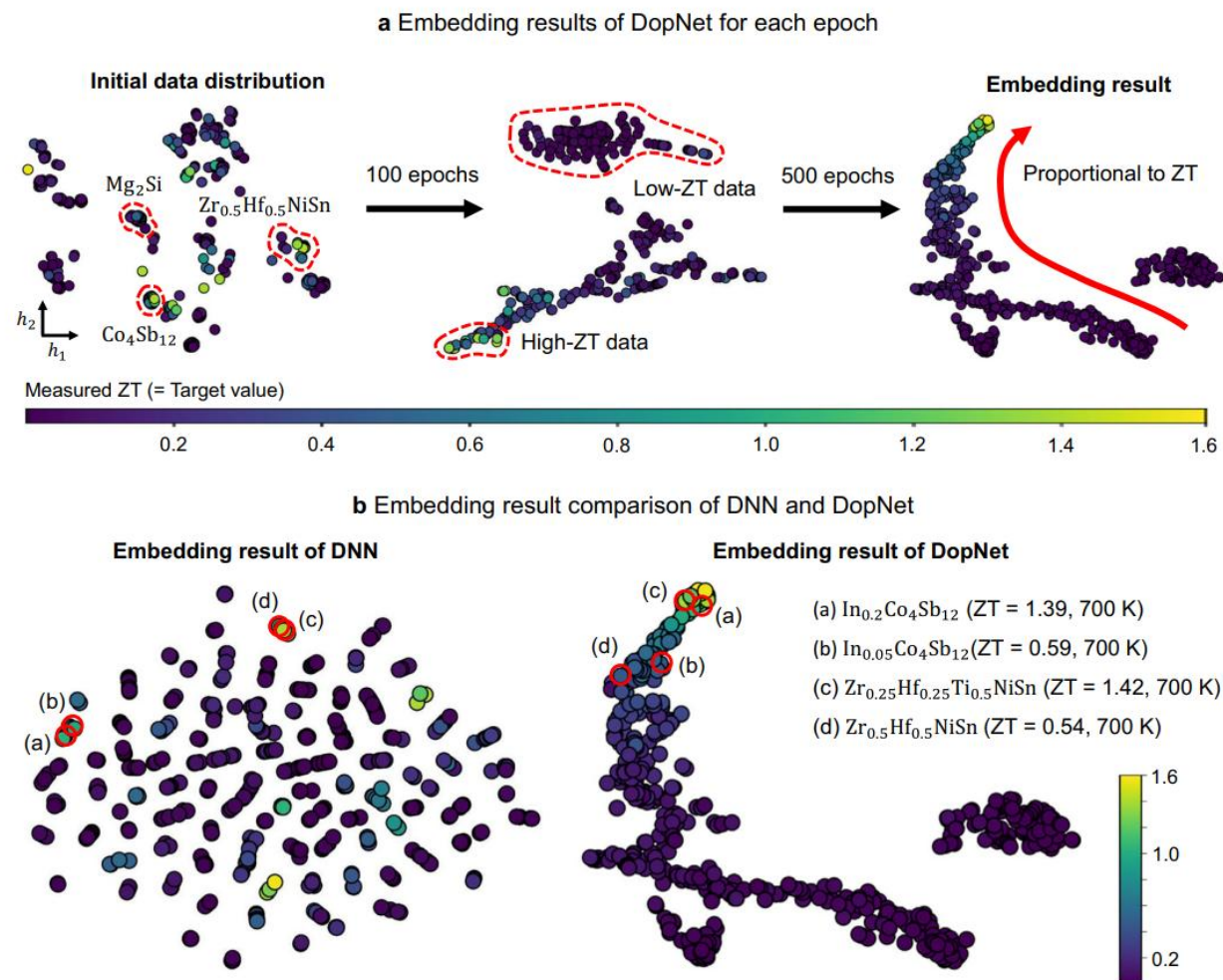
Big-Data Reflection High Energy Electron Diffraction
Analysis for Understanding Epitaxial Film Growth
Processes
ACS Nano 2014, 8, 10, 10899–10908

연구에 사용된 차원 축소(t-SNE) 예시

Predicting thermoelectric properties from chemical formula with explicitly identifying dopant effects

Npj Computational Materials, 2021 7, 106

Gyoung S. Na^{†*}, Seunghun Jang[†], Hyunju Chang
([†] Equally Contributed.)

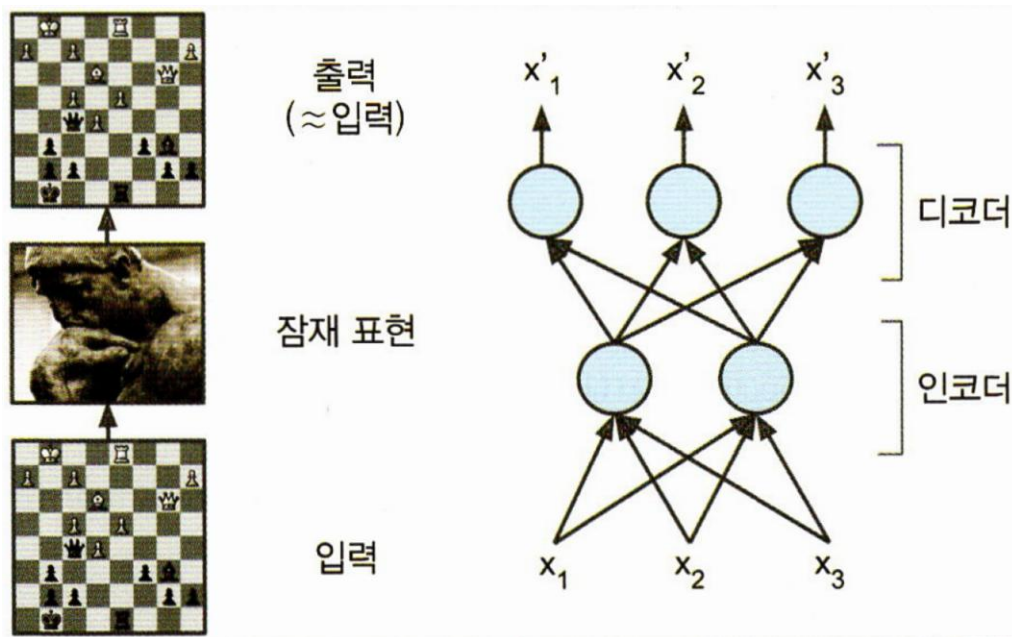


2-dimensional visualization of embedding results of DNN and DopNet

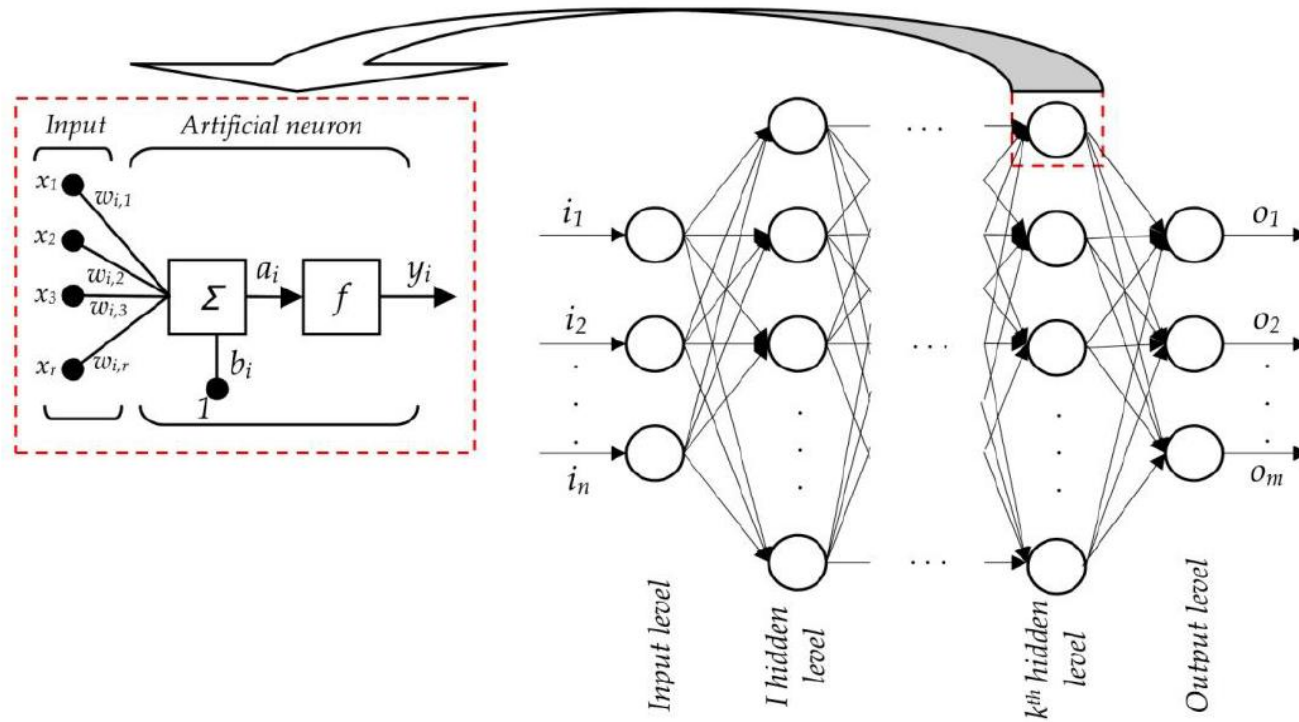
Autoencoder

Autoencoder

- 레이블되어 있지 않은 훈련데이터를 사용하여 잠재표현 또는 코딩이란 부르는 입력데이터의 밀집 표현을 학습할 수 있는 인공 신경망. (**비지도 학습**)
- 입력출력단순 복사학습 또는 일정제약조건하에서 항등함수 학습.
- 입력보다 낮은 차원을 갖게 됨. (시각화에 유용, **차원축소**)
- **일부 오토 인코더**는 훈련데이터와 비슷한 새로운 **데이터를 생성**할 수 있음.



Neural Networks

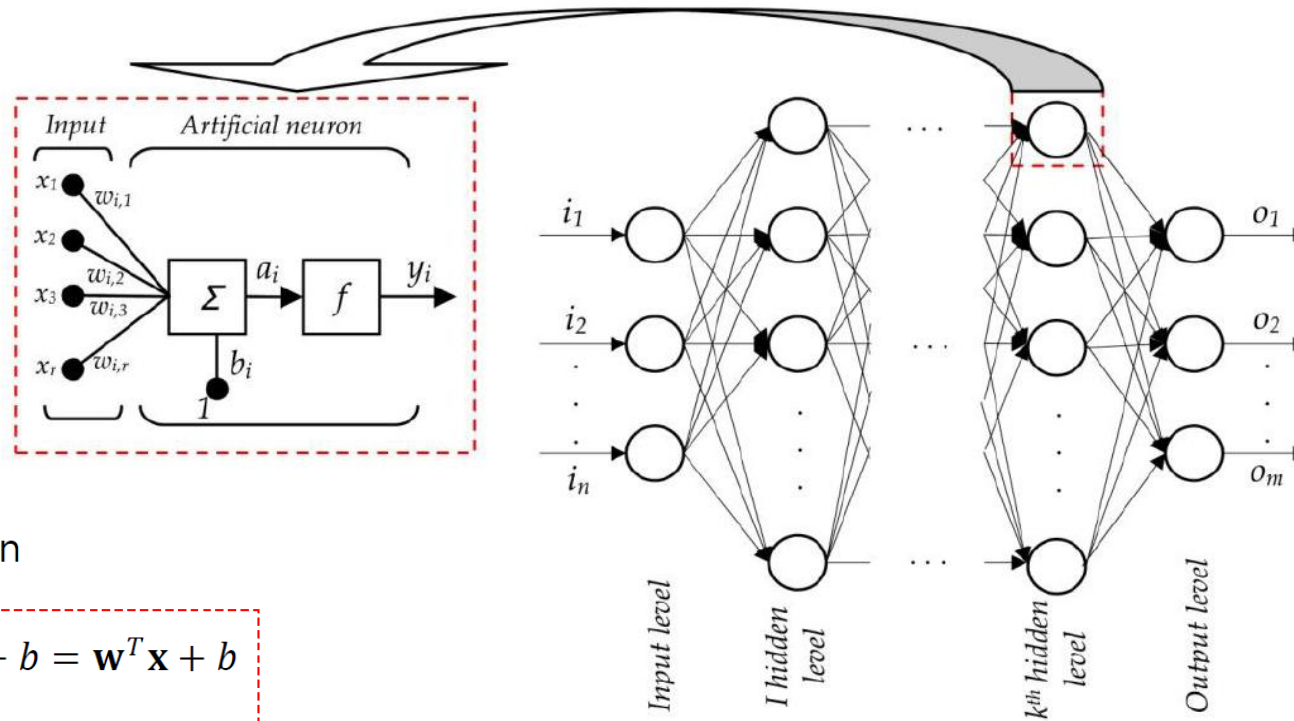


Neural Networks

Layer

$$\boldsymbol{\varphi} = \mathbf{W}^T \mathbf{x} + \mathbf{b} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_m^T \end{bmatrix} \mathbf{x} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

$$\mathbf{y} = f(\boldsymbol{\varphi})$$



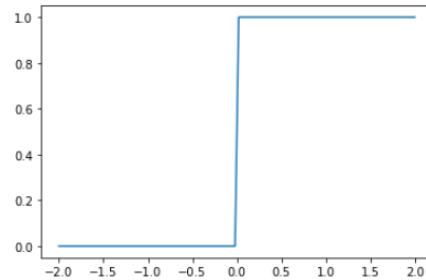
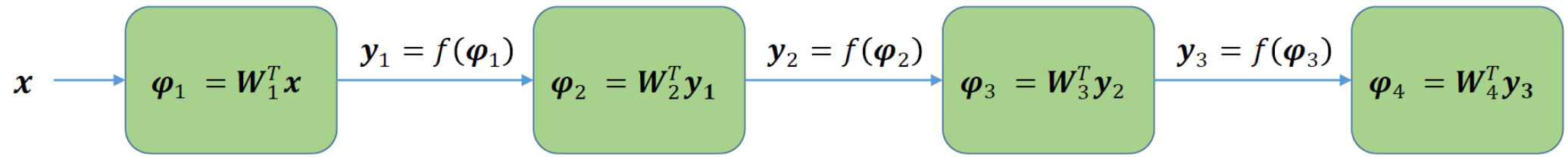
Neuron

$$\varphi = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

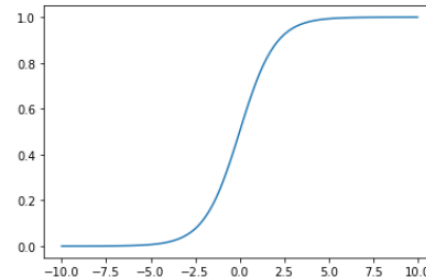
$$y = f(\varphi)$$

Neural Networks

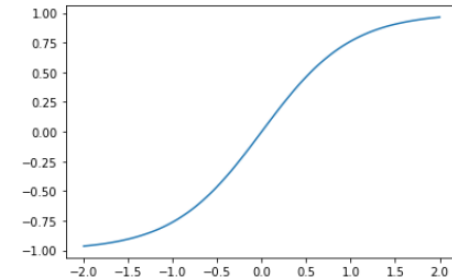
- Non-linear Activation Function, f .
 - Element-wise function.



Step function



sigmoid

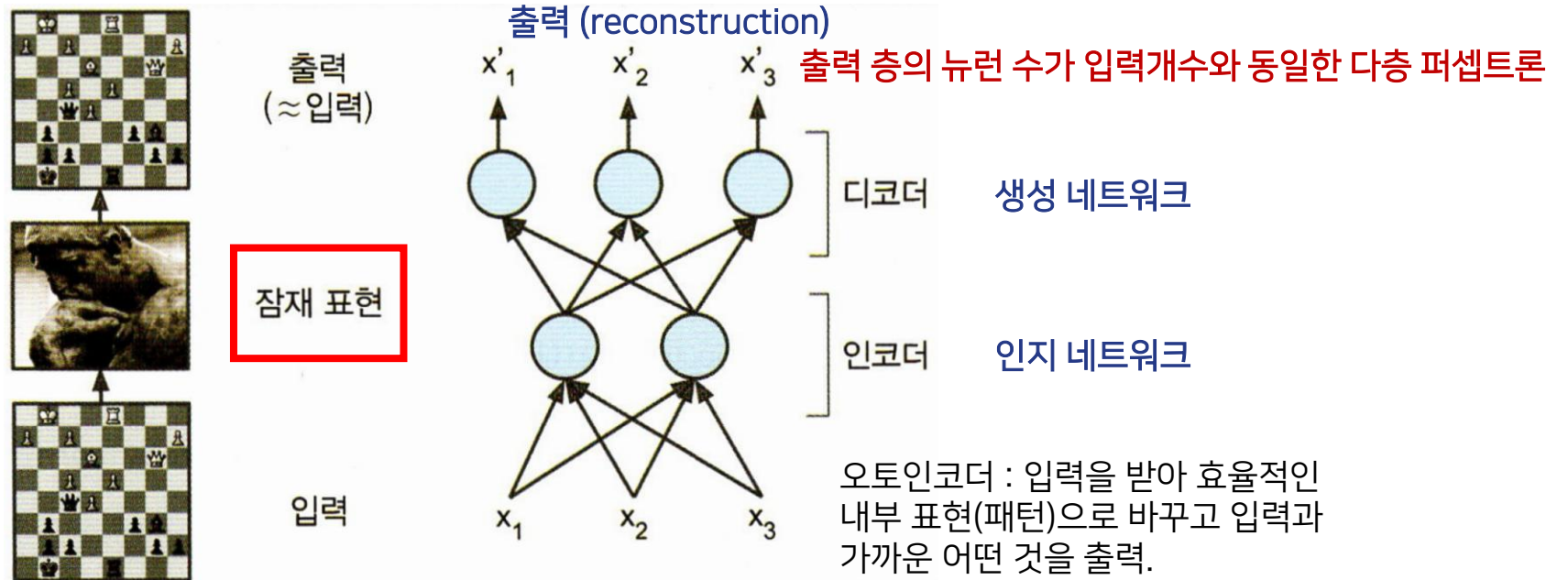


tanh

효율적인 데이터 표현

- 40, 27, 25, 36, 81, 57, 10, 73, 19, 68
- 50, 48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14
- 긴 시퀀스를 기억하기 어려우므로 **패턴**을 찾는 것이 유용.
- **오토인코더에 제약을 가해서** 데이터에 있는 패턴을 찾아 활용.

체스 전문가 : 실제 게임처럼 현실적인 위치에 체스 말들이 있을 때 빠르게 전체 말의 위치를 외움



오토인코더 : 입력을 받아 효율적인 내부 표현(패턴)으로 바꾸고 입력과 가까운 어떤 것을 출력.

Reconstruction error 체크

과소완전 선형 오토인코더로 PCA 수행하기

- 선형 활성화 함수만 사용하고 비용함수가 MSE라면 결국 PCA를 수행하는 것과 동일.
- 단순한 PCA를 수행하기 위해서 활성화 함수를 사용하지 않음 (모든 뉴런이 선형)

```
from tensorflow import keras PCA를 적용하여 2D에 투영하는 간단한 선형 오토인코더
```

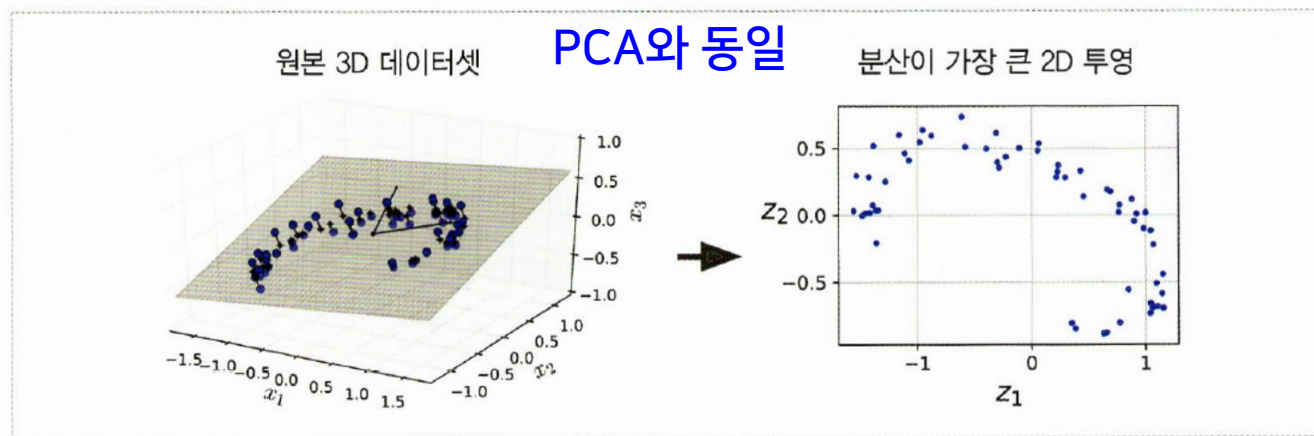
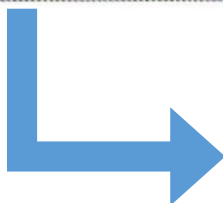
```
encoder = keras.models.Sequential([keras.layers.Dense(2, input_shape=[3])])  
decoder = keras.models.Sequential([keras.layers.Dense(3, input_shape=[2])])  
autoencoder = keras.models.Sequential([encoder, decoder])
```

Dense 층을 가진 일반적인 sequential 모델
Activation function 사용 X → 선형 함수

3 → 2 → 3

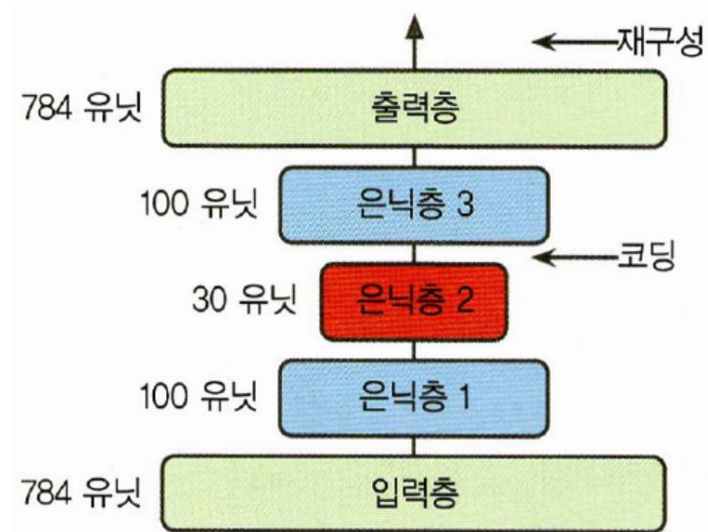
```
autoencoder.compile(loss="mse", optimizer=keras.optimizers.SGD(learning_rate=0.1))
```

```
history = autoencoder.fit(X_train, X_train, epochs=20)  
codings = encoder.predict(X_train)
```



적층 오토인코더

- 오토인코더도 여러 개의 은닉층을 가질 수 있음. (적층인코더 또는 심층인코더)
- 층을 추가할수록 오토인코더가 복잡한 latent feature를 학습.
- 그러나 오토인코더가 너무 강력해도 안됨. (유용한 데이터 표현을 학습하지 못함)
- MNIST input $\rightarrow$ (784-100-30-100-784)



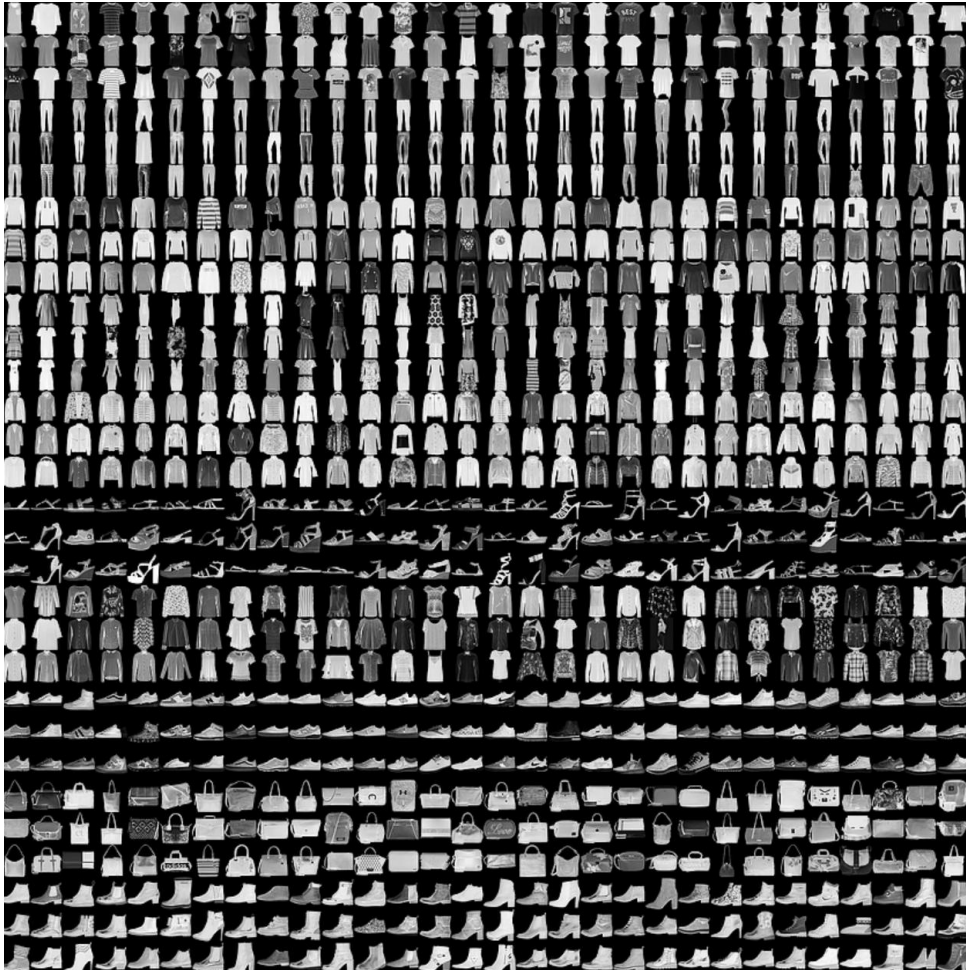
케라스를 사용하여 적층오토인코더 구현하기

- 일반적 심층 MLP와 비슷하게 적층 오토인코더 구현 가능.
- 패션 MNIST 데이터셋, SELU 활성화 함수 사용
- $28 \times 28 \rightarrow 784 \rightarrow 100 \rightarrow 30 \rightarrow 100 \rightarrow 278 \rightarrow 28 \times 28$

```
stacked_encoder = keras.models.Sequential([ 인코더
    keras.layers.Flatten(input_shape=[28, 28]), 28X28 픽셀 인풋을 받아서 784 벡터로 표현
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(30, activation="selu"), SELU 사용
])
stacked_decoder = keras.models.Sequential([ 디코더
    keras.layers.Dense(100, activation="selu", input_shape=[30]),
    keras.layers.Dense(28 * 28, activation="sigmoid"),
    keras.layers.Reshape([28, 28])
])
stacked_ae = keras.models.Sequential([stacked_encoder, stacked_decoder])
stacked_ae.compile(loss="binary_crossentropy",
                    optimizer=keras.optimizers.SGD(learning_rate=1.5))
history = stacked_ae.fit(X_train, X_train, epochs=10,
                        validation_data=(X_valid, X_valid))
```

→ 재구성 작업을 다중레이블 이진 분류(같으면 1 다르면 0) 문제로 다룸.

패션 MNIST 데이터셋



- 패션 MNIST 데이터셋은 위 그림과 같이 운동화, 셔츠, 샌들과 같은 작은 이미지들의 모음
- 기본 MNIST 데이터셋과 같이 열 가지로 분류될 수 있는 28×28 픽셀의 이미지 70,000개로 이루어져 있음

재구성 시각화

- 오토인코더가 적절히 훈련되었는지 확인하는 방법은 입력과 출력을 비교.

```
def plot_image(image):  
    plt.imshow(image, cmap="binary")  
    plt.axis("off")  
  
def show_reconstructions(model, n_images=5):  
    reconstructions = model.predict(X_valid[:n_images])  
    fig = plt.figure(figsize=(n_images * 1.5, 3))  
    for image_index in range(n_images):  
        plt.subplot(2, n_images, 1 + image_index)  
        plot_image(X_valid[image_index])  
        plt.subplot(2, n_images, 1 + n_images + image_index)  
        plot_image(reconstructions[image_index])  
  
show_reconstructions(stacked_ae)
```

Validation 과 reconstruction 을 그리기

Quality of reconstruction image vs informative pattern



원본이미지

재구성이미지

식별이 가능하긴 하지만 정보를 많이 잃어버렸음.

층을 늘리거나 코딩의 크기를 늘려서 모델을 잘 훈련하면 좋을 수 있지만, 반대로 유익한 패턴을 학습하지 못한 채 완벽한 재구성 이미지 만을 만들 수 있음.

패션 MNIST 데이터셋 시각화 AE를 사용한 차원축소

- 데이터셋의 차원축소.

- 다른 차원축소 알고리즘만큼 좋은 결과를 주진 못함
- 그러나 오토인코더는 샘플과 특성이 많은 대용량 데이터셋을 다룰 수 있다는 점이 강점.
- 오토인코더를 사용하여 적절한 수준으로 차원을 축소하고, 이를 다시 다른 차원축소방법으로 축소할 수도 있음. (이중 차원 축소에 활용)

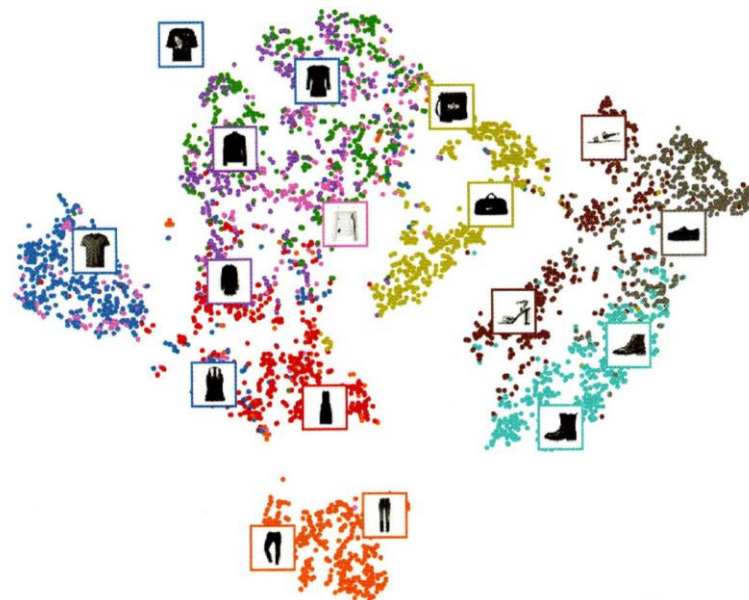
```
from sklearn.manifold import TSNE
```

```
X_valid_compressed = stacked_encoder.predict(X_valid) MNIST-30차원까지 축소
```

```
tsne = TSNE()
```

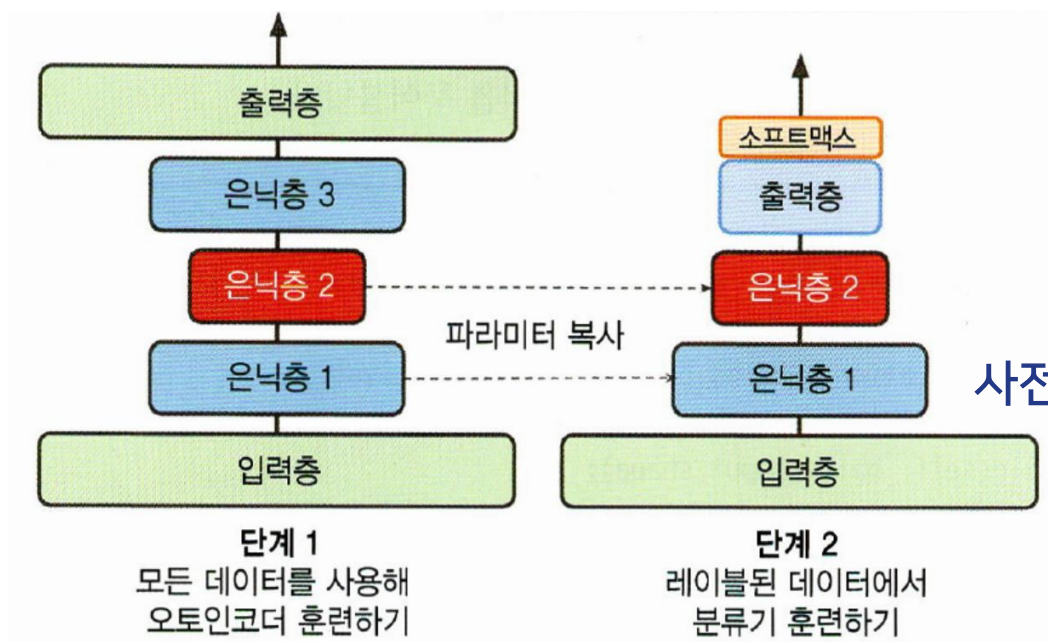
```
X_valid_2D = tsne.fit_transform(X_valid_compressed) t-SNE-2차원으로 축소
```

```
plt.scatter(X_valid_2D[:, 0], X_valid_2D[:, 1], c=y_valid, s=10, cmap="tab10")
```



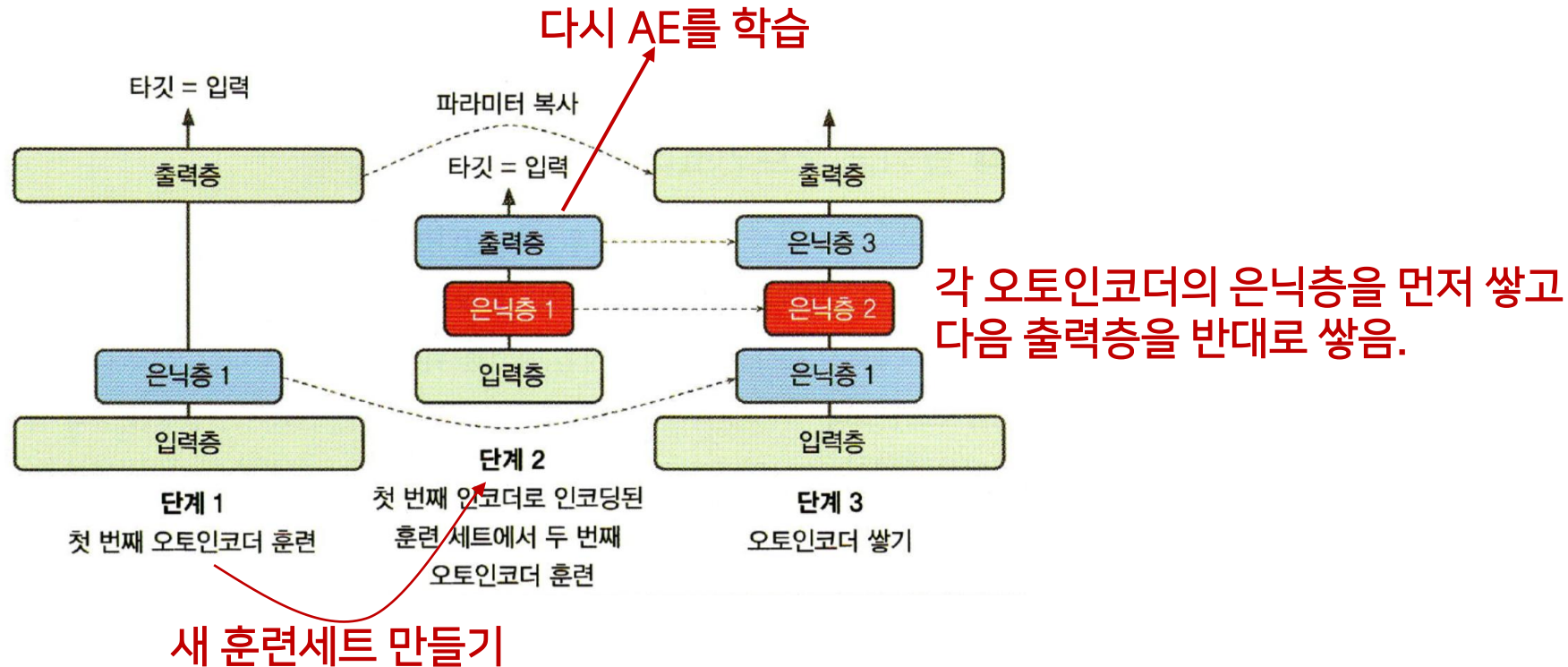
적층 오토인코더를 사용한 비지도 사전훈련

- 레이블된 훈련데이터가 많지 않은 복잡한 지도학습문제의 경우, 비슷한 문제를 학습한 신경망을 찾아 하위층을 재사용하는 것도 좋은 방법. (전이학습 개념)
- 저수준의 특성을 학습할 필요가 없어, 적은 훈련 데이터를 사용해 고성능 모델을 훈련할 수 있음.
- AE를 통해서 찾은 latent feature 를 레이블 데이터에 대한 회귀 문제에 활용.



한 번에 오토인코더 한 개씩 훈련하기

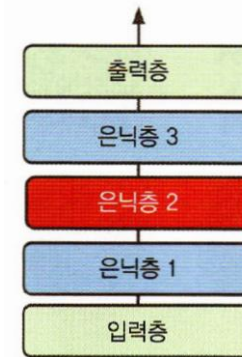
- 한번에 전체 오토인코더를 훈련하는 것보다 아래 그림과 같이 오토인코더 하나를 훈련하고 이를 쌓아 올려서 한 개의 적층 오토인코더를 만들 수 있음.
- 탐욕적 방식의 층별 훈련 (greedy layerwise training)



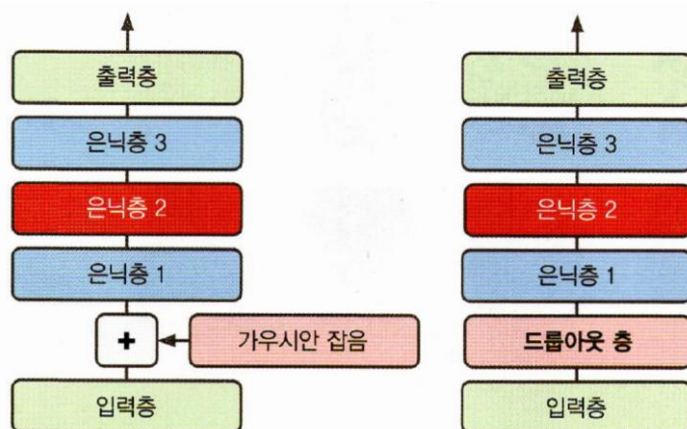
잡음제거 AE

- 잡음을 추가하거나 잡음이 없는 원본 입력을 복원하도록 훈련.
- 얀 르쿤의 1987년 석사 논문에 언급.
- 파스칼 빈센트의 2010년 논문에 적층잡음제거 오토인코더를 소개. Journal of Machine Learning Research 11 (2010): 3371 - 3408

이러한 잡음제거 모델을 잡음(노이즈)이 많은 이미지를 input으로 넣고 적용하면 오른쪽 그림과 같이 출력층을 통해 잡음이 제거된 이미지를 얻게 됨.



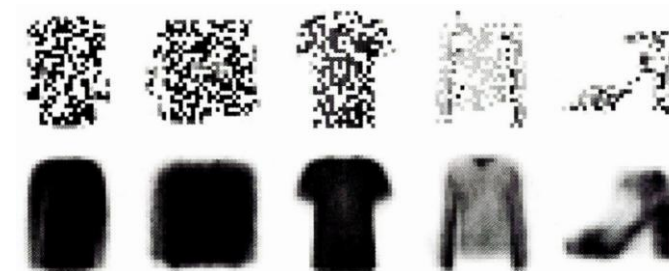
드롭 아웃 층이 있는 일반적인 적층 AE (드롭아웃은 훈련하는 동안에만 활성화, GaussianNoise 층도 마찬가지)



가우시안 잡음(왼쪽) 또는 드롭아웃을 사용한 잡음제거 오토 인코더

```
dropout_encoder = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(30, activation="selu")
])
dropout_decoder = keras.models.Sequential([
    keras.layers.Dense(100, activation="selu", input_shape=[30]),
    keras.layers.Dense(28 * 28, activation="sigmoid"),
    keras.layers.Reshape([28, 28])
])
dropout_ae = keras.models.Sequential([dropout_encoder, dropout_decoder])
```

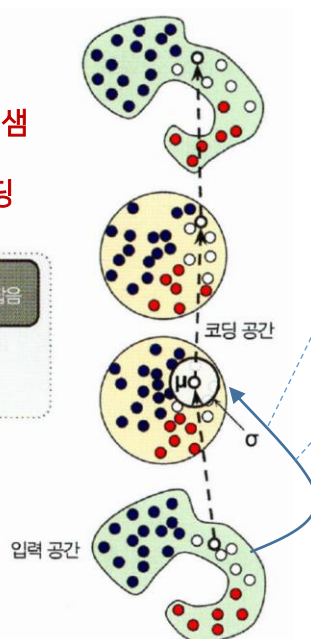
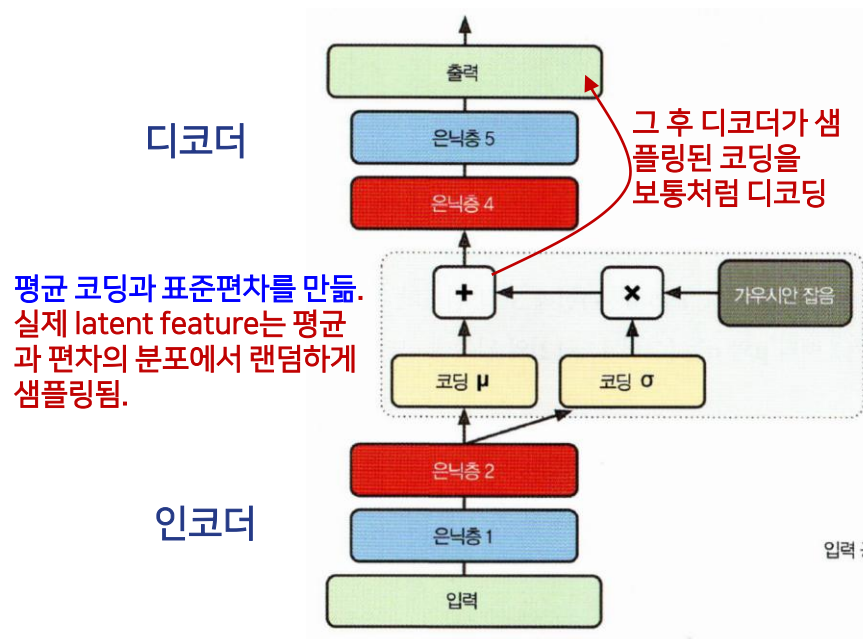
이러한 잡음제거 모델을 잡음(노이즈)이 많은 이미지를 input으로 넣고 적용하면 오른쪽 그림과 같이 출력층을 통해 잡음이 제거된 이미지를 얻게 됨.



잡음 섞인 이미지(위)와 재구성된 이미지(아래)

변이형 오토인코더 (VAE)

- 2014년 다이데릭 킹마와 맥스 웰링이 변이형 오토인코더를 소개했음.
- 확률적 오토인코더. 즉 훈련이 끝난 후에도 출력이 부분적으로 우연에 의해 결정.
- 생성 오토인코더. 마치 훈련 세트에서 샘플링 된 것 같은 새로운 샘플을 생성할 수 있음.
- RBM과 유사하지만 훈련이 더 쉽고 샘플링 과정이 훨씬 빠름.
- 효율적인 근사 베이즈 추론 방법인 변분 베이즈 추론(variational Bayesian inference)

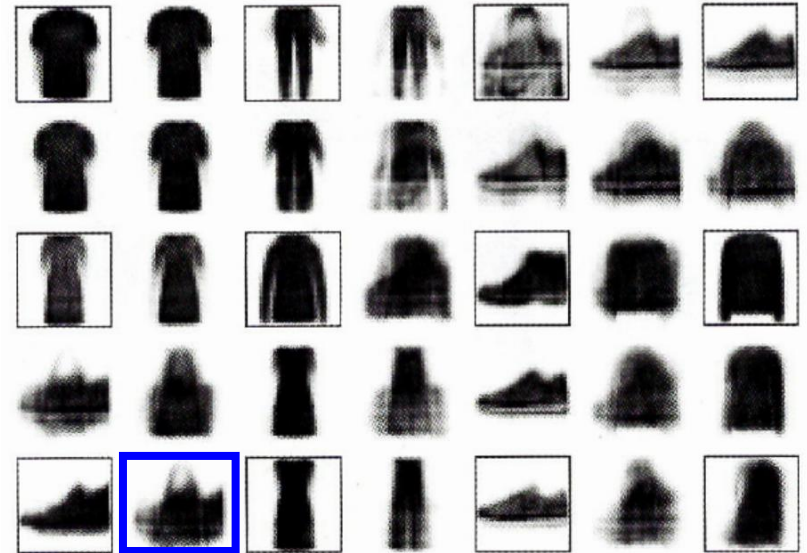
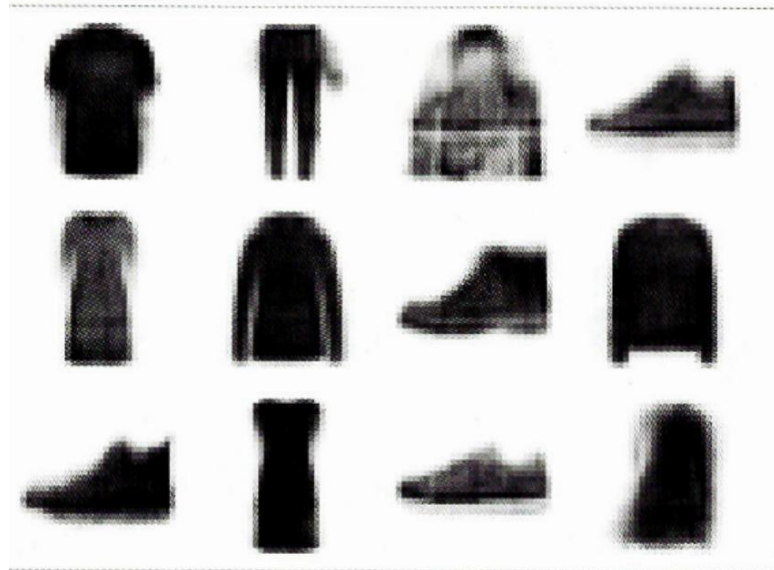


- 입력이 매우 복잡한 분포를 가지더라도 간단한 가우시안 분포에서 샘플링 된 것 처럼 코딩을 만드는 경향이 있음.
- 코딩을 가우시안 샘플들의 군집처럼 보이도록 코딩 공간 안으로 점진적으로 이동
- 따라서 VAE는 훈련이 끝난 뒤 새로운 샘플을 쉽게 생성.

VAE 활용 패션 MNIST 이미지 생성.

- VAE 사용하여 패션의류처럼 보이는 이미지를 생성.
- 가우시안 분포에서 랜덤한 코딩을 샘플링하여 디코딩하는 것이 전부.
- VAE는 시맨틱 **보간(interpolation)** 수행도 가능.
- 지난 몇년간 VAE가 널리 쓰였지만, **GAN**이 훨씬 실제 같이 또렷한 이미지를 만들어줘서 인기가 더 많음.

```
codings = tf.random.normal(shape=[12, codings_size])  
images = variational_decoder(codings).numpy()
```



K-means를 이용한 색상 분할, 전처리, 준지도 학습 & DBSCAN(ChemAI) 실습

Thank you for your attention.
Please feel free to ask questions.

www.ai.chemdx.org
www.simpl-ml.org

jang@kriict.re.kr